Food For Food

“Throwing food away is like stealing from the table of those who are poor and hungry”
-Pope Francis-

Graduation Project by:

Demiana Amgad Farag Yakoob

Fady Salama Fayez Elias

Ibrahim Samir Ibrahim Girgis

Supervisor:
prof. Ibrahim El Awady

Abstraction

"Food for Good" is a mobile application developed using Flutter aimed at reducing food waste and promoting environmental sustainability. The application facilitates the collection of wasted food from restaurants and its delivery to organic fertilizer factories, thereby converting potential waste into valuable resources. This initiative aligns with several United Nations Sustainable Development Goals (SDGs), including responsible consumption and production, climate action, and life on land. By leveraging modern mobile technology, "Food for Good" provides an efficient and scalable solution to the persistent issue of food waste, encouraging restaurants to participate in a circular economy model. The application is designed to streamline the process from waste collection to delivery, ensuring minimal environmental impact while maximizing resource recovery. This project demonstrates how innovative technological solutions can contribute to sustainable development and environmental conservation.

Acknowledgements

We would like to express our sincere gratitude to all those who supported and guided us throughout the development of the "Food for Good" project.

First and foremost, we extend our heartfelt thanks to our advisor, prof. Ibrahim El Awady, for his invaluable guidance, constructive feedback, and continuous encouragement. His expertise and insights were instrumental in shaping this project.

We are also grateful to our professors and lecturers at Assiut University, whose teachings laid the foundation for our knowledge and skills in computer science and mobile application development.

Special thanks to the participating restaurants, truck drivers, and organic fertilizer factories for their cooperation and support during the data gathering and testing phases of this project. Their willingness to engage in this initiative was crucial for its success.

We would also like to acknowledge our peers and classmates for their camaraderie and the stimulating discussions that helped refine our ideas. Their support made this journey more enjoyable and fulfilling.

Lastly, we are deeply indebted to our families and friends for their unwavering support and understanding throughout this process. Their encouragement provided us with the strength and motivation to see this project through to completion.

Table of Contents

1. Introduction

1.1. Background

Food waste is a significant global issue, contributing to environmental degradation and resource inefficiency. Every year, millions of tons of food are wasted, leading to unnecessary greenhouse gas emissions and the depletion of natural resources. Restaurants, in particular, are major contributors to food waste, with large quantities of surplus food often ending up in landfills. In response to this challenge, there is a growing need for innovative solutions that can help reduce food waste and promote sustainability. Our project, "Food for Good," is motivated by the desire to address this issue by creating a mobile application that facilitates the collection and repurposing of wasted food from restaurants into organic fertilizers.

1.2. Problem Statement

The primary problem that "Food for Good" aims to solve is the inefficient management of food waste in the restaurant industry. Currently, most restaurants discard surplus food, which not only contributes to environmental pollution but also represents a missed opportunity to recycle valuable organic material. The lack of a streamlined system for collecting and repurposing this waste exacerbates the problem. Our application seeks to bridge this gap by providing a platform that connects restaurants with organic fertilizer factories via truck drivers, enabling a more sustainable approach to food waste management.

1.3. Objectives

The objectives of the "Food for Good" project are as follows:

1. Develop a user-friendly mobile application using Flutter that facilitates the collection of wasted food from restaurants.
2. Create an efficient logistics system for transporting the collected food to organic fertilizer factories.
3. Promote environmental sustainability by reducing the amount of food waste that ends up in landfills.
4. Raise awareness about the importance of food waste management and encourage restaurants to participate in the initiative.
5. Contribute to achieving several United Nations Sustainable Development Goals (SDGs), including responsible consumption and production, climate action, and life on land.

1.4. Project Scope

The scope of the "Food for Good" project includes the development and deployment of a mobile application designed for restaurant owners, truck drivers, and organic fertilizer factories. The application will cover the following:

- User registration and authentication for restaurants, deliveries, and fertilizer factories.
- A system for restaurants to log and schedule food waste pickups.
- Real-time tracking of food waste collection and delivery.
- Data analytics and reporting features to monitor the impact of the initiative.

The application will initially target a specific geographic region, with potential for future expansion.

1.5. Significance

The "Food for Good" project holds significant potential for positive environmental and social impact. By reducing food waste, the project contributes to the conservation of resources and the reduction of greenhouse gas emissions associated with waste decomposition in landfills. Moreover, repurposing wasted food into organic fertilizers supports sustainable agricultural practices, enriching soil health and reducing the need for chemical fertilizers. The project also fosters collaboration between the food service industry and the agricultural sector, creating a circular economy model that benefits all stakeholders. Ultimately, "Food for Good" demonstrates how technology can be harnessed to address pressing environmental issues and promote a more sustainable future.

2. System Request and Feasibility Analysis

2.1. System Request

2.1.1. Sponsor

Ministry of Health – Environmental Protection Agency (EPA), Green Earth Foundation, Global Food Waste Reduction Initiative

2.1.2. Business Need

We need this application for some reasons such as:

- **Food Waste Reduction:** A significant amount of food is wasted daily, contributing to a global annual waste of approximately 1 billion tons. This project aims to mitigate this issue by creating a mobile application that connects restaurants, organic fertilizer factories, and transportation services. The application will facilitate the collection of surplus food from restaurants and its transportation to factories where it can be converted into organic fertilizer.

- **Employment Opportunities:** The project also aims to reduce unemployment by creating new job opportunities for drivers who will be responsible for transporting the food waste from restaurants to the factories.

2.1.3. Business Requirements:

1. **Restaurant Module:** Restaurants will download the application, register, and log their surplus food. Once a sufficient amount is accumulated, they will use the application to find the nearest available driver. The application will also identify the factory that requires the specified amount of food waste.

2. **Driver Module:** Drivers will receive notification of pickup requests from nearby restaurants via the application. They will be responsible for collecting the food waste and delivering it to the designated factory.

3. **Factory Module:** Factories will receive food waste from the drivers, convert it into organic fertilizer, and process the payment to the application's account for the delivered waste.

2.1.4. Business Value:

1. **Reduction of Food Waste:** The application will contribute to a significant decrease in the amount of food wasted globally.

2. **Job Creation:** The project will create new job opportunities for drivers, reducing unemployment.

3. **Boost to Fertilizer Industry:** By providing a steady supply of organic material, the project will increase the production of organic fertilizers.

4. **Revenue Generation:** The project will generate income through:
    - **Selling Wasted Food:** Restaurants will sell their surplus food to factories.
    - **Advertisements:** Revenue from ads displayed within the application.

2.2. Feasibility Analysis

2.2.1. Technical Feasibility

Familiarity with User

The development team has conducted thorough research to understand the needs and preferences of the users, including restaurant owners, drivers, and factory managers.

Familiarity with Technology

The team is well-versed in using Flutter for mobile application development, as well as other relevant technologies and frameworks.

Project Size

The project is manageable in scope, with well-defined features and milestones.

Duration: 6 months
Number of Developers: 3 persons
Working Environment: hybrid

Compatibility

The application is designed to integrate seamlessly with existing systems used by restaurants and factories. The only requirement is to be connected to the internet to use the application.

2.2.2. Economic Feasibility

	Year 0	Year 1	Year 2	Year 3	Year 4	Total
Benefits						
Advertisements	-	$15700	$20200	$25300	$38500	$99700
Organic Fertilizer	-	$57300	$98800	$124700	$161500	$442300
Total Benefits	-	$73000	$119000	$150000	$200000	$542000
Development Costs						
Drivers	$30000	-	-	-	-	$30000

Software	$35000	$5000	$5000	$5000	$5000	$55000
Restaurants	$5000					$5000
Total Development Cost	$70000	$5000	$5000	$5000	$5000	$90000
Operational Costs						
Drivers	-	$48000	$92000	$120000	$132000	$392000
Application	-	$8250	$8250	$8250	$8250	$33000
Total Operational Costs	-	$56250	$100250	$128250	$140250	$425000
Total Costs	$70000	$61250	$105250	$133250	$145250	$515000
PV of Total Benefits ($B/1.1^Y$)	-	$66363	$98347	$116697	$138602	$420010
PV of Total Costs ($C/1.1^Y$)	$70000	$55681	$85983	$99112	$108128	$418904
Net Benefits (Total Benefits – Total Costs)	$(70000)	$11750	$13750	$16750	$54750	$27000
Cumulative Net Cash Flow	$(70000)	$(58250)	$(44500)	$(27750)	$27000	
Return On Investment (ROI)	((Total Benefits−Total Costs)/Total Costs)*100 ((542000 - 515000)/ 515000) *100 = 5.24%					
Break Even Point (BEP)	$BEP = \text{Number of years of negative cash flow} + \frac{\text{That year's Net Cash Flow} - \text{That year's Cumulative Cash Flow}}{\text{That year's Net Cash Flow}}$ 3 + ((54750 - 27000)/ 54750) = 3.5					
Present Value (PV)	$(70000/(1.1)^4) = $47810.94					
Net Present Value (NPV)	PV of Total Benefits - PV of Total Costs = 420010 - 418904 = $1106					

Ultimately, the financial analysis of the project spans four years, highlighting a trajectory from initial investment to profitability. Starting with development costs totaling $90,000, including expenditures on drivers, software, and initial setup, the project incurred operational costs of $425,000 over the period. Despite an initial loss of $70,000 in Year 0, benefits from advertisements and organic fertilizer sales grew annually to reach $542,000 by Year 4. This growth translated into increasing net benefits, turning positive in Year 1 and reaching $54,750 by Year 4. The Return On Investment (ROI) calculated at 5.24% indicates the project's profitability relative to costs, with a Break Even Point (BEP) achieved in Year 3.5. Present Value (PV) of costs after 4 years is $47,810.94, and the Net Present Value (NPV) stands at $1,106, showing the project's profitability considering the time value of money. Overall, the project demonstrates a promising financial outlook, with a clear path to profitability and positive returns on investment.

2.2.3. Organizational Feasibility

Low Risk: From an organizational perspective, this project carries low risk due to its alignment with current sustainability initiatives and the support from multiple sponsors and stakeholders.

Strong Sponsor Interest: The project is backed by multiple sponsors, including the Ministry of Health, the Environmental Protection Agency (EPA), the Green Earth Foundation, and the Global Food Waste Reduction Initiative. These sponsors have a vested interest in the success of the project, providing strong financial and strategic support.

Project Champion: The project has a dedicated champion within the sponsoring organizations, ensuring continuous advocacy and alignment with organizational goals and sustainability initiatives.

User Appreciation: The primary users of the system—factories, restaurants, and drivers—are expected to appreciate the application due to its potential to streamline operations, reduce costs, and create new revenue streams. Restaurants will benefit from reduced waste disposal costs and potentially increased income from selling surplus food. Drivers will gain new job opportunities, and factories will have a reliable source of organic material for fertilizer production.

Management Concerns: While management at the factories may have some initial concerns about integrating the application into their existing operations, the lack of alternative solutions for efficiently sourcing organic waste positions our system as a valuable resource. This system may prevent the loss of factory partnerships to competitors and provide a strategic advantage in securing a steady supply of raw materials.

Stakeholder Support:

There is strong support from key stakeholders, including:

- **Restaurants:** Enthusiastic about reducing waste and potentially generating additional income.
- **Organic Fertilizer Factories:** Interested in securing a consistent supply of organic material.
- **Logistics Partners:** Potential partners in the transportation sector see value in the increased demand for delivery services.

Resource Availability:

The necessary human and material resources are readily available:

- **Human Resources:** The project team consists of skilled developers, testers, project managers, and support staff with expertise in mobile application development and sustainability projects.
- **Material Resources:** Adequate infrastructure, including servers, hosting services, and development tools, are in place to support the application's development and deployment.

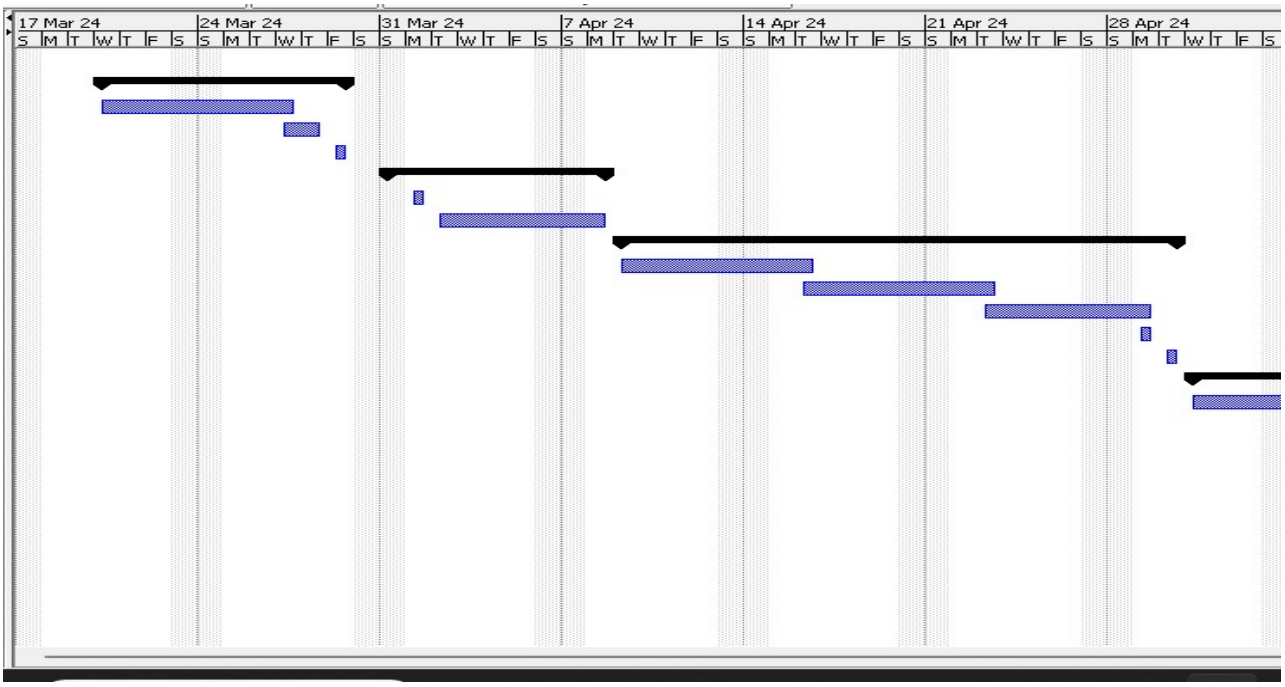
2.2.4. Project Timeline Plan

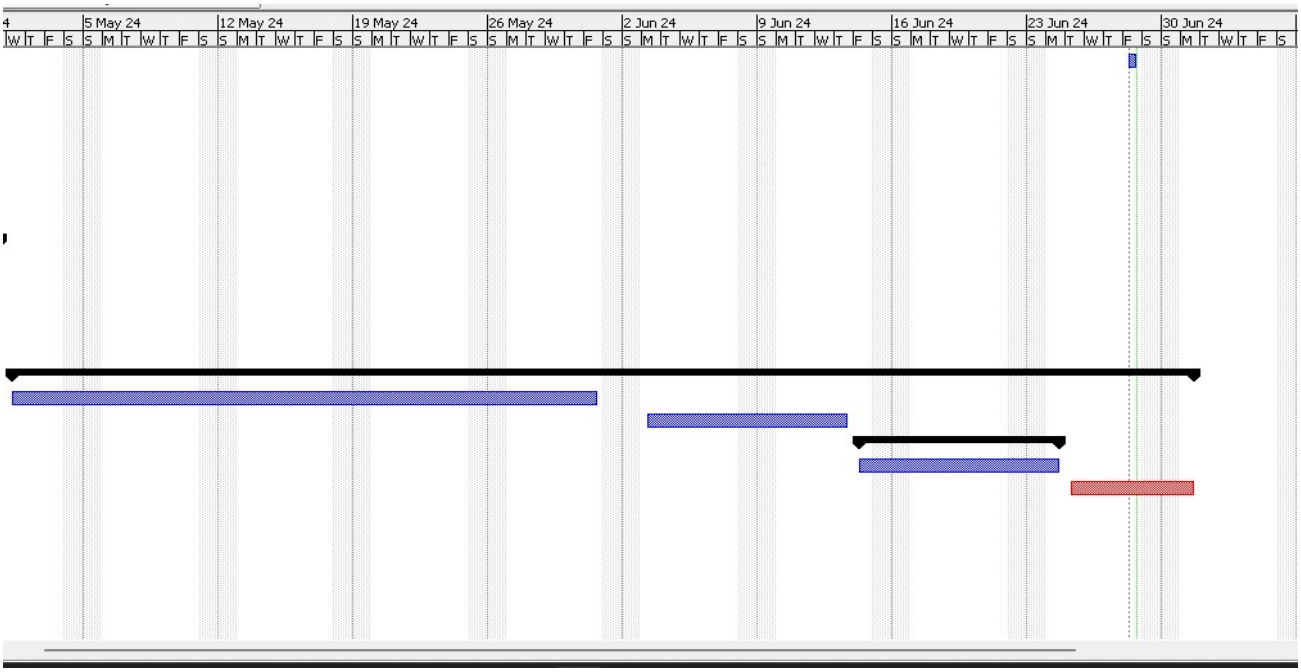
We utilized ProjectLibre to construct the Timeline Plan for our graduation project. Employing this tool was instrumental in ensuring meticulous time management throughout our project lifecycle. ProjectLibre enabled us to visualize and structure our project timeline effectively, allowing us to allocate resources efficiently and meet crucial milestones in a timely manner. This approach not only facilitated a clear understanding of project dependencies and deadlines but also empowered us to maintain focus on our project objectives and deliverables. By leveraging ProjectLibre, we could proactively monitor progress and make informed adjustments to our timeline as needed, thereby optimizing our project's overall efficiency and success.

Project Timeline

		Name	Duration	Start	Finish
4		System requeest	1 day?	6/28/24 8:00 AM	6/28/24 5:00 PM
5		**Fesability Analysis**	**8 days?**	**3/20/24 8:00 AM**	**3/29/24 5:00 PM**
6		Technical Feasibilty	6 days?	3/20/24 8:00 AM	3/27/24 5:00 PM
7		Economicl feasibility	2 days?	3/27/24 8:00 AM	3/28/24 5:00 PM
8		Organizational Feasibility	1 day?	3/29/24 8:00 AM	3/29/24 5:00 PM
9		**Analysis Phase**	**6 days?**	**3/31/24 8:00 AM**	**4/8/24 5:00 PM**
10		System needs	1 day?	3/31/24 8:00 AM	4/1/24 5:00 PM
11		Project Requirements	5 days?	4/2/24 8:00 AM	4/8/24 5:00 PM
12		**Design Phase**	**16 days?**	**4/9/24 8:00 AM**	**4/30/24 5:00 PM**
13		Use cases	6 days?	4/9/24 8:00 AM	4/16/24 5:00 PM
14		DFD and ERD	6 days?	4/16/24 8:00 AM	4/23/24 5:00 PM
15		Sequence diagram	5 days?	4/23/24 8:00 AM	4/29/24 5:00 PM
16		Functional requirements	1 day?	4/29/24 8:00 AM	4/29/24 5:00 PM
17		non-functional requirement	1 day?	4/30/24 8:00 AM	4/30/24 5:00 PM
18		**Development**	**44 days?**	**5/1/24 8:00 AM**	**7/1/24 5:00 PM**
19		Coding	23 days?	5/1/24 8:00 AM	5/31/24 5:00 PM
20		Optimizing code	9 days?	6/1/24 8:00 AM	6/13/24 5:00 PM
21		**Testing**	**7 days?**	**6/14/24 8:00 AM**	**6/24/24 5:00 PM**
22		Integration and testing	7 days?	6/14/24 8:00 AM	6/24/24 5:00 PM
23		Documentation	5 days?	6/25/24 8:00 AM	7/1/24 5:00 PM

Project schedule





3. Literature Review

3.1. Existing Research and Applications

The issue of food waste is a critical global problem that has garnered significant attention in academic and industry research. Several studies highlight the environmental, economic, and social impacts of food waste, emphasizing the need for sustainable solutions.

1. **Food Waste Statistics and Environmental Impact:**
   - According to the Food and Agriculture Organization (FAO), approximately one-third of all food produced globally is wasted annually, amounting to about 1.3 billion tons.
   - The environmental impact of food waste is substantial, contributing to 8-10% of global greenhouse gas emissions.
2. **Existing Applications and Initiatives:**
   - **Too Good To Go:** An application that connects consumers with restaurants and stores that have unsold surplus food. This app focuses on reducing food waste by offering discounted meals to consumers.
   - **Olio:** A community-based app that allows individuals and businesses to share surplus food with neighbors and local communities, aiming to reduce household food waste.
   - **Food Rescue US:** This platform focuses on redistributing excess food from restaurants and grocers to food-insecure communities via volunteer drivers.

3.2. Gaps in Current Solutions

Despite the efforts of existing applications, there are notable gaps that "Food for Good" aims to address such as:

Sustainable Revenue Model:

1. Unlike apps that rely heavily on donations or discounted sales to consumers, "Food for Good" introduces a sustainable revenue model by facilitating transactions between restaurants and fertilizer factories. This model ensures a continuous and reliable income stream for all parties involved.

4. Methodology

4.1. Research Questions

4.1.1. Questions for restaurants managers

- Are you able to use the mobile application easily?
- Do you support the concept of the application?

- What is the daily amount of wasted food in your restaurant?
- How do you manage the wasted food in your restaurant?
- What challenges do you anticipate while using the application?
- Will you consider stopping the donation of your wasted food in the future?
- How would you feel if we informed you that we will collect this wasted food for a nominal fee?
- What price would you expect to receive for your food?
- How would you respond if you were informed that there are no drivers or factories available to collect your wasted food?
- Would you prefer to arrange for drivers to collect the food, or should we provide this service for you?

4.1.2. Questions for factory managers

- How familiar are you with the concept of repurposing wasted food into organic fertilizers?
- As a factory manager, how often does your facility require organic materials like those derived from repurposed food waste?
- How do you currently source organic materials for your production needs?
- Would your factory be interested in receiving wasted food collected from restaurants for the production of organic fertilizers?
- What challenges, if any, do you foresee in utilizing wasted food as a raw material for organic fertilizers? (Select all that apply)
- How important is sustainability and environmental responsibility to your factory's operations?
- Would your factory be willing to pay a nominal fee for the collection and transportation of wasted food from nearby restaurants?
- How likely are you to participate in a program that promotes sustainable practices such as repurposing wasted food into organic fertilizers?
- What additional features or services would you like to see in the "Food for Good" application to facilitate your participation?
- Any other comments or suggestions regarding the concept of repurposing wasted food for organic fertilizers?

4.1.3. Questions for drivers

- Are you currently employed as a professional driver?
- How familiar are you with the concept of transporting wasted food from restaurants to organic fertilizer factories?
- How often do you engage in transportation services involving perishable goods or food items?
- Would you be interested in participating in a program that involves transporting wasted food from restaurants to organic fertilizer factories?

- What factors would influence your decision to participate as a driver in such a program? (Select all that apply)
- How important is environmental sustainability to you personally?
- What challenges, if any, do you anticipate in transporting wasted food from restaurants to organic fertilizer factories?
- Would you prefer to have a dedicated mobile application to manage pickups and deliveries of wasted food?
- How likely are you to recommend this program to other drivers or colleagues?
- Any additional features or services you would like to see in the "Food for Good" application to facilitate your role as a driver?
- Any other comments or suggestions regarding the concept of transporting wasted food for repurposing into organic fertilizers?

4.2. Interviews

4.2.1. Interviews with Restaurants Managers

Interview Schedule

Name	Phone NO	Restaurant	Position	Purpose of interview
Mohamed Nady	01025353026	Afandina (fishes)	Manager	To know how the app is useful for them and their satisfactions
Khaled Hosny	010095564537	BOOM (fried chicken)	Manager	To know how the app is useful for them and their satisfactions
Ahmed Abdegmil	01010666516	Crinkle (fried chicken)	Co-manager	To know how the app is useful for them and their satisfactions
Mamdoh Shokry	01091312402	Sablee (Pastries)	Co-manager	To know how the app is useful for them and their satisfactions
Anonymous	Anonymous	El menshawy(Kushary)	Manager	To know how the app is useful for them and their satisfactions
Anonymous	Anonymous	Sultana restaurant	Manager	To know how the app is useful for them and their satisfactions

Interview Details

Interview with Mohamed Nady
Interviewee's position: Manager Purpose of the interview: To know how the app is useful for them and their satisfactions. Summary of interview • It is easy to use the application

- One problem will face us: the wasted food contains rubbish, and it is not easy to collect pure wasted food
- The amount of wasted food became less than before due to COVID pandemic.
- We need to increase payment to the restaurant if we need the wasted food without rubbish.
- No problems for the restaurant to give us their wasted food, but it is hard to keep the food there more than one day.

Interview with Khaled Hosny
Interviewee's position: Manager Purpose of the interview: To know how the app is useful for them and their satisfactions. Summary of interview • It is easy to use the app • No problems to give us their wasted food • The restaurant gives good, wasted food for poor people and through the rest out. • The manager refused the payment for the wasted food because he does not need it at all. • Manager refused to store the wasted food in the restaurant.

Interview with Ahmed Abdegmil
Interviewee's position: Co-Manager Purpose of the interview: To know how the app is useful for them and their satisfactions. Summary of interview • It is easy to use the app. • No problems giving us their wasted food, but it will contain rubbish. • No problems taking money for the wasted food. • No ability to store the wasted food

Interview with Mamdoh Shokry
Interviewee's position: Co-Manager Purpose of the interview: To know how the app is useful for them and their satisfactions. Summary of interview • No problems helping us if it will help the community. • Poor people take the wasted food. • No problems giving us their wasted food. • Refused to take money for the wasted food. • Provides drivers for us if we do not have

Interview with El-Minshawy
Interviewee's position: Manager Purpose of the interview: To know how the app is useful for them and their satisfactions. Summary of interview • Support our idea. • No problems giving us the wasted food, but some people take it to their birds. • The amount of wasted food depends on how many people visit us per day.

• No ability to store the wasted food in the restaurant.

Sultana restaurant
Interviewee's position: Assistant Manager Purpose of the interview: • To know how the app is useful for them and their satisfactions. • The main thing is to understand their point of view of the application to re-manage our project if necessary. Summary of the interview: Since the idea of application is encouraging in some restaurants such as this one, then we will continue with it, but clearly some points: • This application is very easy to use and helpful. • Lots of restaurants want us to bring the driver because they don't need to be responsible for them. • We should take food daily to not be damaged. • Restaurants can help us when drivers don't exist but not all the time.

4.2.2. Interviews with Restaurants Managers

Interview Schedule

Name	Position	Purpose of interview
Ali Ahmed	Manager	knowing more details about factories like which of them need the wasted food. knowing about their opinion on our application. Let us know which points need to be re-think of it.
Momen Salah	Manager	Knowing more details about factories' needs. Collecting Ideas to improve user experience. Deciding the best way to work with the factory.

Interview Details

Ali Ahmed
Summary of the interview: • Factories support our idea, but their opinion is that our app gives them unstable in quantities. • They will take the wasted food even if they have enough quantity but on their terms. • They will stop taking the wasted food from us if the quantity becomes less. • If the fertilizer becomes more expensive, they will pay more but if it will be less, they will pay less than before. • When we asked them about our app, they told us that it has advantages and disadvantages. • They will be able to use the app if we provide them with a video to help them.

Momen Salah

Summary of the interview: • Factories support our idea only in the case of providing constant quantities. • A desktop or web application will be more suitable for the factories. • Daily delivery is the best option. • The factories will not handle the delivery.

4.2.3. Interviews with Drivers

Interview Schedule

Name	Position	Purpose of interview
Mohsen Kamal	Driver	Knowing more details about drivers' needs. Collecting Ideas to improve user experience. Deciding the best way to work with
Aseel	Truck owner	To Collect information about the driver's needs. To know what the impact of our idea on the driver is.

Interview Details

Mohsen Kamal
• Drivers support our idea. • A mobile application will be more suitable for the divers. • Taking the salary daily according to the quantity and location is the preferred option for drivers. • Drivers prefer to work with us in their free time.

Aseel
After explained our idea to the driver and made him know most of the details that he want, I find that: • The driver liked our idea. • I make sure that our application is easy to use like most of the other applications. • The driver can deal with transferring the wasted food wisely

4.3. Overview of the Approach

The "Food for Good" project follows a structured approach, leveraging modern mobile technology to provide a scalable solution to food waste management. The methodology includes several stages: design and development of the mobile application, data collection, implementation, and evaluation.

- **Development Approach**: The project uses the Agile development methodology, which involves iterative and incremental development. Agile allows for flexibility and adaptability in the development process, accommodating changes and feedback throughout the project's lifecycle.

4.4. Data Collection Methods

Data collection for this project involves both primary and secondary sources:

- **Primary Data**: Collected through surveys and interviews with restaurant owners and staff to understand their practices regarding food waste and their willingness to participate in the initiative.
- **Secondary Data**: Gathered from existing literature on food waste management, environmental impact studies, and case studies of similar initiatives.

4.5. Tools and Technologies Used

- **Development Framework**: Flutter, for building the cross-platform mobile application.
- **Backend Services**: Firebase for real-time data synchronization and MongoDB for secure database management were selected to meet the project's needs.
- **APIs**: Google Maps API, for route optimization and tracking delivery paths.
- **Analytics Tools**: Google Analytics, for monitoring user engagement and application performance.

4.6. Data Analysis Techniques

- **Qualitative Analysis**: Content analysis of interview transcripts to identify common themes and insights.
- **Quantitative Analysis**: Statistical analysis of survey data to measure the extent of food waste and the potential impact of the application.
- **Environmental Impact Assessment**: Calculating the reduction in carbon footprint achieved by diverting food waste from landfills to organic fertilizer production.

4.7. Implementation Brief Overview (technology stack)

- Design and Development
  - **Frontend**: Creating a user-friendly interface using Flutter's design principles.
    - **Framework:** Flutter
    - **Programming Languages:** Dart
    - **UI/UX Tools:** Android studio, Visual Studio Code
  - **Backend Integration (data management)**: Setting up server for authentication, data storage, and real-time updates.
    - **Framework:** Node.js, Express.js for authentication
    - **Programming Languages:** JavaScript, TypeScript
    - **Database:** MongoDB (NoSQL database)
    - **APIs:** Enabling real-time updates via RESTful APIs
  - **API Integration**: Incorporating Google Maps API for efficient routing and tracking.
- Testing
  - **Unit Testing:** Ensuring individual components of the application function correctly.
  - **Integration Testing**: Verifying that different parts of the application work together seamlessly.
  - **User Acceptance Testing (UAT)**: Collecting feedback from potential users to refine the application
- Deployment
  - Launching the application on Google Play Store and Apple App Store.
  - Promoting the application to potential users, primarily targeting restaurants

4.8. Challenges and Limitations

- Data Privacy: Ensuring the protection of user data and compliance with relevant data protection regulations.
- Restaurant Participation: Overcoming resistance from restaurants to change their existing waste management practices.
- Logistical Coordination: Managing the logistics of food collection and delivery to ensure efficiency and minimize environmental impact.

4.9. Ethical Considerations

- Informed Consent: Obtaining consent from all participants involved in surveys and interviews.
- Data Security: Implementing robust security measures to protect user data.
- Environmental Responsibility: Ensuring that the project adheres to sustainable practices and promotes genuine environmental benefits.

5. System Analysis and Design

5.1. Functional and Non-functional requirements

5.1.1. Functional Requirements

1. **User Registration and Authentication**
    a. Users (restaurants) must be able to register with the application using their email or social media accounts.
    b. Users must be able to log in and log out securely.
2. **Food Waste Listing**
    a. Restaurants must be able to list available food waste, including details such as type, quantity, and expiration date.
    b. Users must be able to edit or delete their listings
3. **Collection Scheduling**
    a. Users must be able to schedule collection times for the food waste.
    b. The application must notify users of the scheduled collection times.
4. **Route Optimization**
    a. The system must optimize collection routes using the Google Maps API to ensure efficient pickups.
5. **Delivery Tracking**
    a. The application must allow users to track the status of their food waste collection and delivery to organic fertilizer factories.
6. **Notifications**
    a. The system must send notifications to users about upcoming collection times, status updates, and important information.
7. **User Feedback**
    a. Users must be able to provide feedback on the collection and delivery process.
8. **Reports and Analytics**
    a. The application must generate reports on the amount of food waste collected and converted into fertilizer.
    b. Users must have access to analytics regarding their contributions to food waste reduction.

5.1.2. Non-functional Requirements

1. **Performance**
    a. The system must handle up to 10,000 concurrent users without performance degradation.
    b. Response time for user requests must be less than 2 seconds.
2. **Scalability**
    a. The application must be scalable to accommodate an increasing number of users and food waste listings.

3. **Security**
    a. User data must be encrypted during storage and transmission.
    b. The system must implement robust authentication and authorization mechanisms to prevent unauthorized access.
4. **Usability**
    a. The application must have an intuitive and user-friendly interface.
    b. The design must be accessible to users with varying levels of technical proficiency.
5. **Reliability**
    a. The application must have an uptime of 99.9%.
    b. The system must implement error-handling mechanisms to ensure continuity of service.
6. **Maintainability**
    a. The codebase must be well-documented to facilitate maintenance and future updates.
    b. The system architecture must support easy integration of new features and improvements.
7. **Compatibility**
    a. The application must be compatible with both Android and iOS platforms.
    b. The system must support various screen sizes and device specifications.
8. **Compliance**
    a. The application must comply with relevant data protection and privacy regulations.
    b. The system must adhere to industry standards for mobile application development.

5.2. Use Case Diagrams

5.2.1. Login and Sign-up

Use case name: Sign up	ID: UC1	Priority: High
Actor: Users use the application for the first time		
Description: users create an account to use the application		
Trigger: restaurant manager wants to gain money from the wasted food, factory manager wants to buy wasted food, or a driver wants to work as a delivery		
Type: External		
Preconditions: • the user opened the app and it worked with them without errors and then pressed on the sign-up button on the screen. • The user has available contact • The user has available bank account or Instapay account		

Normal Course		Information for steps
1. The user press on sign up.	→	Sign up page will be opened
2. The system shows welcome message and open signing up page.		
3. The user enters the first name	←	First name will be taken
4. The user enters the last name.	←	Last name will be taken
5. The user enters the birth date.	←	Birth date will be taken
6. The user chooses their gender.	←	Gender will be taken
7. The user enters their phone number.	←	Phone number will be taken
8. The system asks the user to write the confirmation code that is sent on an SMS		
9. The user writes the confirmation message.	←	Confirm phone number
10. The user enters their email.	←	Email will be taken
11. The user enters the account number	←	Account number will be taken

12. The system asks the user to choose whether they are restaurant manager, factory manager, or a truck driver		
13. The user chooses the suitable choice.	←	Managers/drivers will be known
14. The user press on "continue" button to finish creating the account.	→	License and password page will be opened
15. The system asks the user to scan needed licenses.		
16. The user scans the needed licenses.		
17. The system asks the user to enter a strong password.	←	Licenses will be sent to admins
18. The user enters the password.		
19. The user enters the password again to be confirmed	←	Password will be taken
20. The user enters on "I am not robot" sauare.	←	Password will be confirmed
21. The user press on “Create account” button	→	Account will be created
22. Account will be created, and licenses will be sent to be checked by admins		
Alternative course		Information for steps
1. The user has invalid bank account or instapay **1.1** the system will ask the user to activate an account **1.2** the system will exit from the sign-up, but data will be saved	← →	Licenses will be sent to admins Passwords will be saved
2. The user does not have a bank account **2.1** The system will ask the user if he accept to take his money cash from the factory	←	Request message
3. The user has chosen whether they are driver or factory manager or restaurant manager **3.1** The user is driver (in step 13):		

3.1.1 The system asks the user to scan the driving license and the car license. **3.2** The user is restaurant manager (in step 13): **3.2.1** The system asks the user to enter the restaurant license. **3.3** The user is factory manager (in step 13): **3.3.1** The system asks the user to enter the factory license.	←	Request message Request message Request message
4. The user entered incorrect phone number **4.1** The system shows the message "phone number must consist of 11 numbers"	←	Request message
5. the user entered weak password **5.1** The system show message "enter strong password" (strong password must contain letters, numbers, and special letters)	←	Request message
6. If the user entered used email or phone number **6.1** The system suggests to the user to sign in or exit and show the message "this email or phone number is used before, try to sign in **6.1.1** The system asks the user to enter their email and password. **6.1.2** The user enters the email address and	← →	Request message email and password will be taken

their password **6.2** The user exit from the application		
7. (in log in) the user entered incorrect password **7.1** The system asks the user to reenter the password (available to enter incorrect password only for 5 times) **7.1.1** the user reenters the password **7.1.2** the user reenters the password **7.1.2.1** the system asks to user to enter the code which is sent to their phone number or email address **7.1.2.2** The user enters the code and open the account		

Exceptions:

1. The user entered invalid birth date (for drivers, less than the licensed age).
2. The user entered banned bank account.
3. The user entered invalid phone number.
4. The user entered expired licenses.
5. The users disconnected from the internet so they can't use the app.
6. The user changed their mail so they can't confirm his information by the code number.

Post Conditions:

1. The user has a valid account with email and strong password.
2. The user able to use the app.

Summary inputs	Sources	Summary Outputs	Destination
First name Last name Date of birth Phone number Confirmation msg number	User	Sign up page Welcome message Requests messages	New user

Email Licenses Password Sign in code			

5.2.2. Restaurant Announcement

Use case name: Restaurant Announcement		ID: UC2	Priority: Medium
Actor: Restaurant Managers			
Description: The managers want to announce that they has x amount of wasted food, and they want a driver to transport it			
Trigger: The manager wants to get rid of the wasted food on their restaurant and get profit			
Type: External			
Preconditions: The user has a valid account with a verified email and a strong password. The user can use the app. The user is logged in.			
Normal Course			Information for steps
1. The manager opens the app as a restaurant in signing up step.	→		Welcome message
2. The manager presses on the announcement button.			
3. The manager announces that he has an amount of wasted food.	←		Know the wasted food amount
4. The manager shows which factory is nearby to them.	→		Nearby factories will appear
5. The manager shows if there are available drivers or not.	→		Nearby drivers will appear
6. The manager presses on the driver's icon who is nearby from them.	←		Driver and manager will communicate
7. The manager will communicate with the driver to take the wasted food			
8. When these steps will be done, the process will be succeeded.	→		Succeeded message

Alternative course		
1. The manager sign up but not as restaurant (server error) 1.1 The manager will refresh the app.		
2. The manager announce a wrong quantity of wasted food 2.1 If factories want a different quantity (more or less) so that they didn't go to this restaurant, and they will wait for a different factory with a suitable quantity		
3. the manager didn't find available drivers 3.1 The manager will communicate with factories to solve this problem across the chat		

Exceptions:
1. The manager did not find drivers at all.
2. There are no factories need food at this time (the food will be through away).

Post Conditions:
1. The manager got rid of the wasted food.
2. The manager got money from this wasted food.
3. Free drivers make the best benefit of their free time.

Summary inputs	Sources	Summary Outputs	Destination
The wasted food amount	User	Welcome message Nearby factories Nearby drivers Succeeded message	Manager

5.2.3. Factory user

Use case name: Factory User	ID: UC3	Priority: high
Actor: Factory manager or who use the app from any factory.		
Description: When Factories need some amount of wasted food , they will use the app to announce about their need of wasted food by taking them from the App's drivers to use them usefully in their field and give the app the fee of the amount that he need.		
Trigger: Users need the enough amount of wasted food and pay for the app according to the weight of wasted food.		
Type: External		
Preconditions: 1. The user must open the app and it work with him without errors and then press on the 'Log In' button on the screen. 2. If the user doesn't have an account, he should sign up to create an account. 3. The app agreed to work with this factory to ensure good dealings with him. 4. The user must have a bank account (Visa card) or mezza card to pay to the app the suitable fee by it. 5. The factory should need wasted food to use them. 6. The user should know how to use the app or any technology.		

Normal Course		Information for steps
1. The user should open the app.		
2. The system shows welcome message for users.	←	Opening the app
3. The app will recognize the type or major of the user as we said before in "sign up use case" so it will show the factory his suitable form.		
4. For the first time the app will explain how to use the app professionally.	←	Get to know the program
5. The user will enter his phone number that he wants from the app to communicate with him when the order arrives,	←	Start to use the program
6. The user will enter the date and the time that he needs to work through.	←	Choosing suitable needs

7. 7- The app will show the available weights and kind of foods that can be given to factories per day.		
8. The user will choose the amount of food that he needs.		
9. The user will choose the kind of food that he wants.		
10. The user needs to set the average usage date of this food in order to determine the production and expiry date of the product that he will produce.		
11. The app will tell the factory's user about the fee that he should pay for his amount of wasted food.		
12. If the user accepted to pay the money, the app will show a message confirmation about the information that he chose before.	←	Confirmation about payment
13. The app will show to the user the time taken from the arrival of his request.	←	How long it will take to get order
14. The app will send a message to the user that it takes his money successfully.		
15. The app will tell the user the available time that he could cancel the order if he had any problem but after this time, he can't cancel the order.	←	Time of canceling order
16. The user can Chat With the driver Who Will give him his order (we will take about how to chat in other use case).	←	Chatting with the driver
17. When the order arrived, the driver will call the user by the number that he entered before.		
18. The app will appear a message to the user that he got his order to confirm the process successfully.	←	confirmation about success the Process
19. The app will ask the user about his evaluation about the service.		
20. The app will thank the user.		
21. The user will exit from the app.	←	Close the app

22.		
Alternative course		
1. The user didn't focus with the first explanation of how to use the app or forget any step: 1.1-the system will have an explanation button (in Step 4)	←	Misunderstanding
2. he user entered wrong phone number to communicate with the driver 2.1-The app will send a verify code to ensure about your phone number (in step 5). 2.2-the system shows the message "phone number must be 11numbers" If it is different from 11 number.	←	Issue with phone number
3. The user will choose wrong amount of food that he needs or wrong kind of food that he wants. (In Step 8 & 9) 3.1-he factory will tell the app in the evaluation part about the problem. 3.2-The factory would face this problem if it happened again.	←	Wrong information
4. The account bank of the factory is empty (in step 11&12) 4.1-The app will show a message says" You don't have enough money, try again late 4.2-The system will give him a choice to pay cash.	←	Empty card
5. The app didn't show to the user the time taken from the arrival of his request. (In step 13) 5.1-The user can call the driver b the driver's information in the app.	←	Unknown arrival time
6. When the order arrived, the driver didn't call the user by the number that he entered before. (in step 17) 6.1-he user should tell the app that he is late, and the app will communicate with the driver and tell the reason or the issue.	←	Driver didn't call factory
7. The app won't appear a message to the user that he got his order to confirm the process successfully. (in step 18) 7.1-The user can press on "Done" button.	←	Didn't confirm the process
Exceptions: 1-There is no wasted food for the time that the user need. 2- The user will enter wrong date and the time that he needs to work through. 3- The user did not find drivers at all. 4- The application in updating mode. 5- The time of canceling order is finished.		

6- The account or the card that the user will use to pay to the app is closed for bank issues or any problem,			
Post Conditions: 1. The factory will use this wasted food to serve the community by saving the environment healthier 2. Free drivers make the best benefit of their free time. 3. The factory will gain lots of many by manufacturing organic fertilizer. 4. The user will make organic fertilizer healthier than in the past.			
Summary inputs	Sources	Summary Outputs	Destination
User number User free date Users need amount of wasted food Users' kind of food	User	welcome message, Users take amount of food that they need. Evaluation	User App

5.2.4. Scheduling

Use case name:	ID: UC4	Priority: High
Actor: Driver		
Description: The driver will log in to the application to announce their available work hours by choosing the time they will be available.		
Trigger: The driver decides to announce their available work hours.		
Type: External		
Preconditions: The user has a valid account with a verified email and a strong password. The user can use the app. The user is logged in.		

Normal Course		Information for steps
1. The application opens the main page.	→	The main page is displayed.
2. The driver navigates to the schedule section	→	The schedule page is opened
3. The driver presses the schedule button	→	The schedule page is displayed
4. The driver enters the desired work hours.	←	The entered work hours are recorded.
5. The driver enters their current contact information.	←	The contact is recorded
6. The application admin sends a verification code to the provided contact information.	→	A verification code is sent to the driver's contact.
7. The driver enters the received verification code	→	The code is verified.
8. The driver presses the submit button	←	The entered information is submitted and the driver receives a confirmation message.
Alternative course		
1. Invalid Contact Information 1.1 The driver enters their current contact information 1.2 The application admin attempts to send a	←	The contact information is recorded.

verification code to the provided contact information. 1.3 The driver is notified that the contact information is invalid and is prompted to re-enter their contact information. 1.4 The driver enters a new contact information. 1.5 The application admin sends a verification code to the new contact information. 1.6 The driver enters the received verification code. 1.7 The driver presses the submit button.	←	The new contact information is recorded.

Exceptions:
1. The driver may enter an unavailable or incorrect contact information

Post Conditions:
1. The driver's available work hours are successfully added to the application schedule.
2. The driver's available work hours are successfully added to the application schedule after entering valid contact information.

Summary inputs	Sources	Summary Outputs	Destination
Available work hours Current contact information Verification code	User	Schedule page update Confirmation message to the driver	Schedule page Handling the time with drivers

5.2.5. Chatting

Use case name: Chatting	ID: UC5	Priority: High
Actor: Drivers, Restaurant Managers, Factory Managers		
Description: Facilitate communication between drivers, restaurant managers, and factory managers to coordinate the transport of wasted food from restaurants to factories.		
Trigger: Chatting among the driver, the restaurant manager, and the factory manager.		
Type: Temporal		
Preconditions: 1. The driver, restaurant manager, and factory manager must sign in to the app. 2. The app must be operational without any errors.		
Normal Course		Information for steps
1. The factory manager searches for a restaurant with the required amount of wasted food.		
2. The restaurant manager sends a message to an available driver with the restaurant's location.		
3. The driver picks up the food from the restaurant and delivers it to the factory.		
Alternative course		
1. If there is an error in the app's internet connection, the best way to communicate between the factory manager, the driver, and the restaurant manager is by phone calls.		
Exceptions: 1. Messages sent by the factory, restaurant, or driver may not arrive due to issues such as internet disconnection. 2. Delays in the restaurant's response to the driver's messages may cause some losses.		
Post Conditions: 1. The driver, restaurant manager, and factory manager will fill in their daily information (availability, suitable distance for the driver, amount of wasted food available for the restaurant and required by the factory, etc.) and complete their processes. 2. The driver will see announcements from restaurants needing drivers to transport wasted food		

and from factories needing drivers to deliver wasted food. 3. The driver will choose the best opportunities to start their work. 4. The restaurant and factory managers will receive information about the driver and the payment method. 5. Communication between the restaurant manager, the driver, and the factory manager will be successful, ensuring the food is delivered from the restaurant to the factory.

Summary inputs	Sources	Summary Outputs	Destination
Message Location	Restaurant Manager Factory Manager	Response from Drivers	User (Driver, Restaurant Manager, Factory Manager)

5.2.6. Delivery process

Use case name:	ID: UC6	Priority: High
Actor: Driver		
Description: This use case describes the process by which a driver picks up wasted food from a restaurant and delivers it to a factory.		
Trigger: A factory requests wasted food, or a restaurant announces it has wasted food to be delivered.		
Type: External		
Preconditions: 1. The driver must have a valid account. 2. There is a valid request from a factory. 3. There is enough wasted food in a restaurant near the driver.		

Normal Course		Information for steps
1. The driver announces availability		New driver available.
2. The driver sends their location to the application.	←	Driver's location recorded.
3. The application sends the restaurant location to the driver	→	Restaurant location received.
4. The driver drives to the restaurant and picks up the wasted food		Delivery started

5. The application sends the factory location to the driver	→	Factory location received.
6. The driver delivers the wasted food to the factory.		Delivery finished.
Alternative course		
1. The restaurant is very far from the driver. 1.1 The driver can refuse the request and ask for another location.	→	New restaurant location sent
2. The driver doesn't have enough space for the wasted food. 2.1 The application will send the driver to another location.	→	New factory location sent.

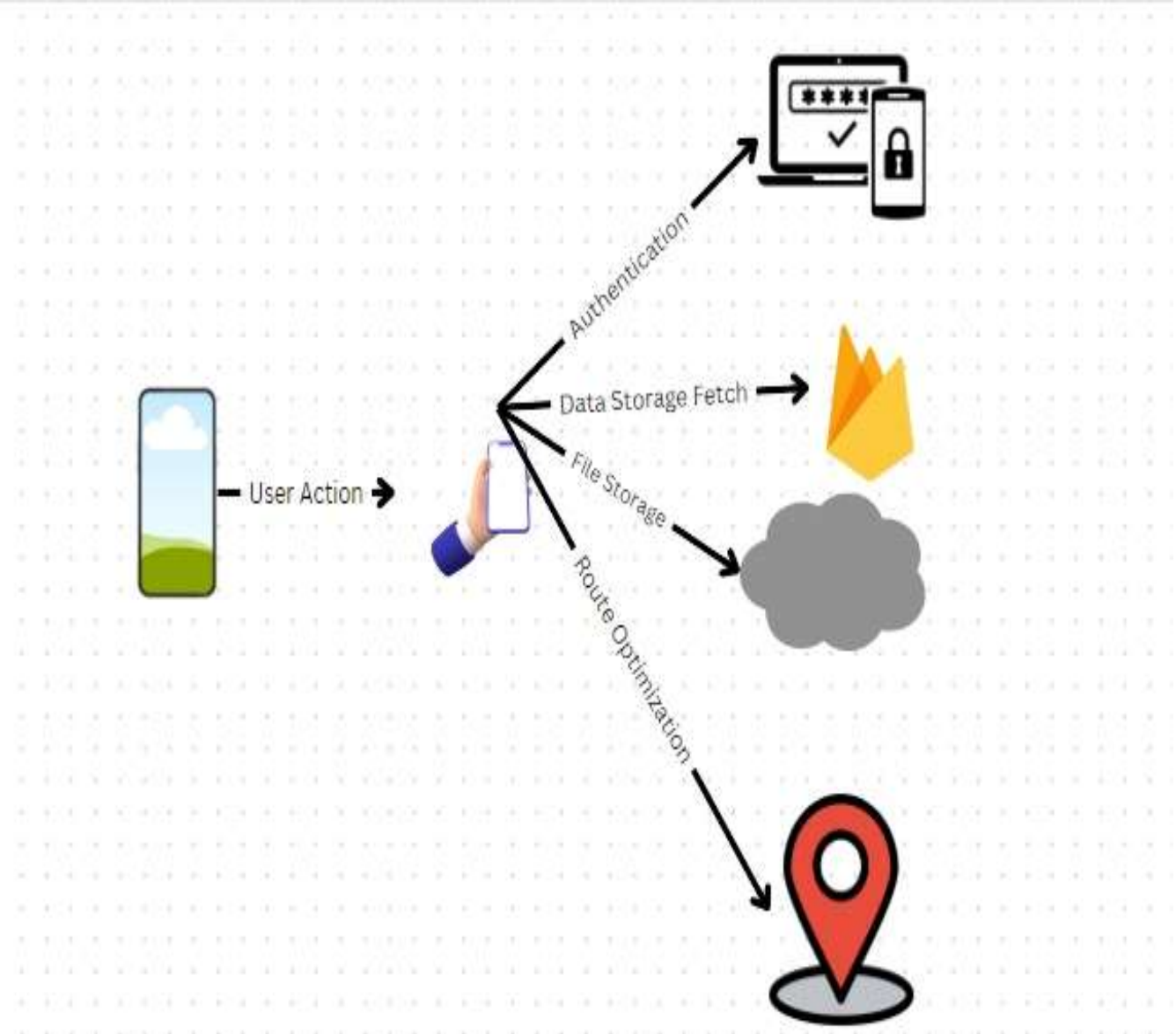
Exceptions:
1. The driver's account isn't valid (missing information).
2. The driver is too far from any restaurants or factories.
3. The driver doesn't have a stable internet connection.
4. There is no wasted food available to deliver.
5. There aren't any factory requests for a delivery.

Post Conditions:
1. The driver gets paid.
2. The restaurant gets paid.
3. The delivery process is recorded in the application database.

Summary inputs	Sources	Summary Outputs	Destination
Driver's location Restaurant's location Factory's location	Driver	Restaurant's location Factory's location	Driver

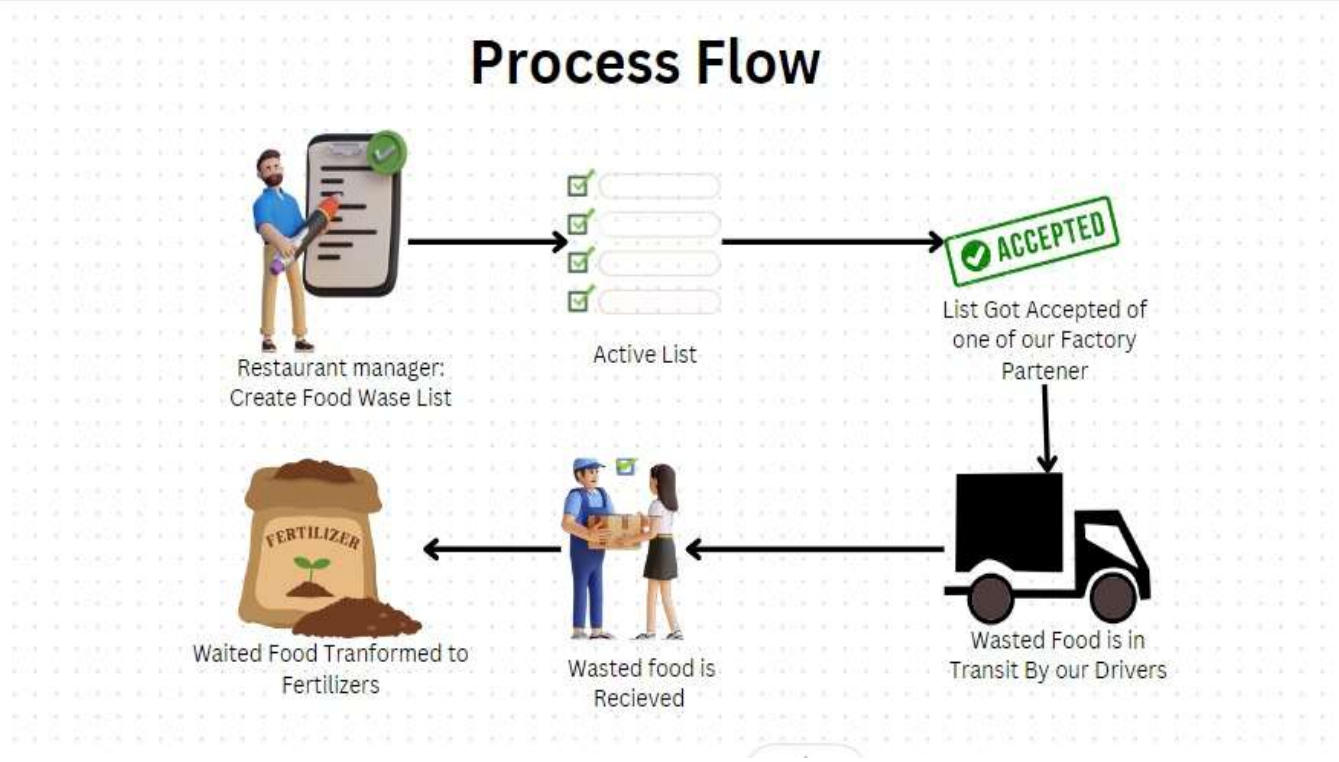
5.3. System Architecture Diagram

The diagram illustrates the high-level architecture of the "Food for Good" mobile application. It shows the interactions between different system components, focusing on how data flows and services interact.



5.4. Process flow Diagram

The diagram effectively maps out the lifecycle of a food waste listing within the "Food for Good" application. It begins with the creation and submission of a listing, making it active and available for other users to view and accept. Once accepted, the food waste is picked up and transported, ultimately being delivered to its destination. Each transition is clearly labeled, highlighting the actions that move a listing from one state to the next. This structured flow ensures clarity in understanding how the application handles food waste listings from start to finish.



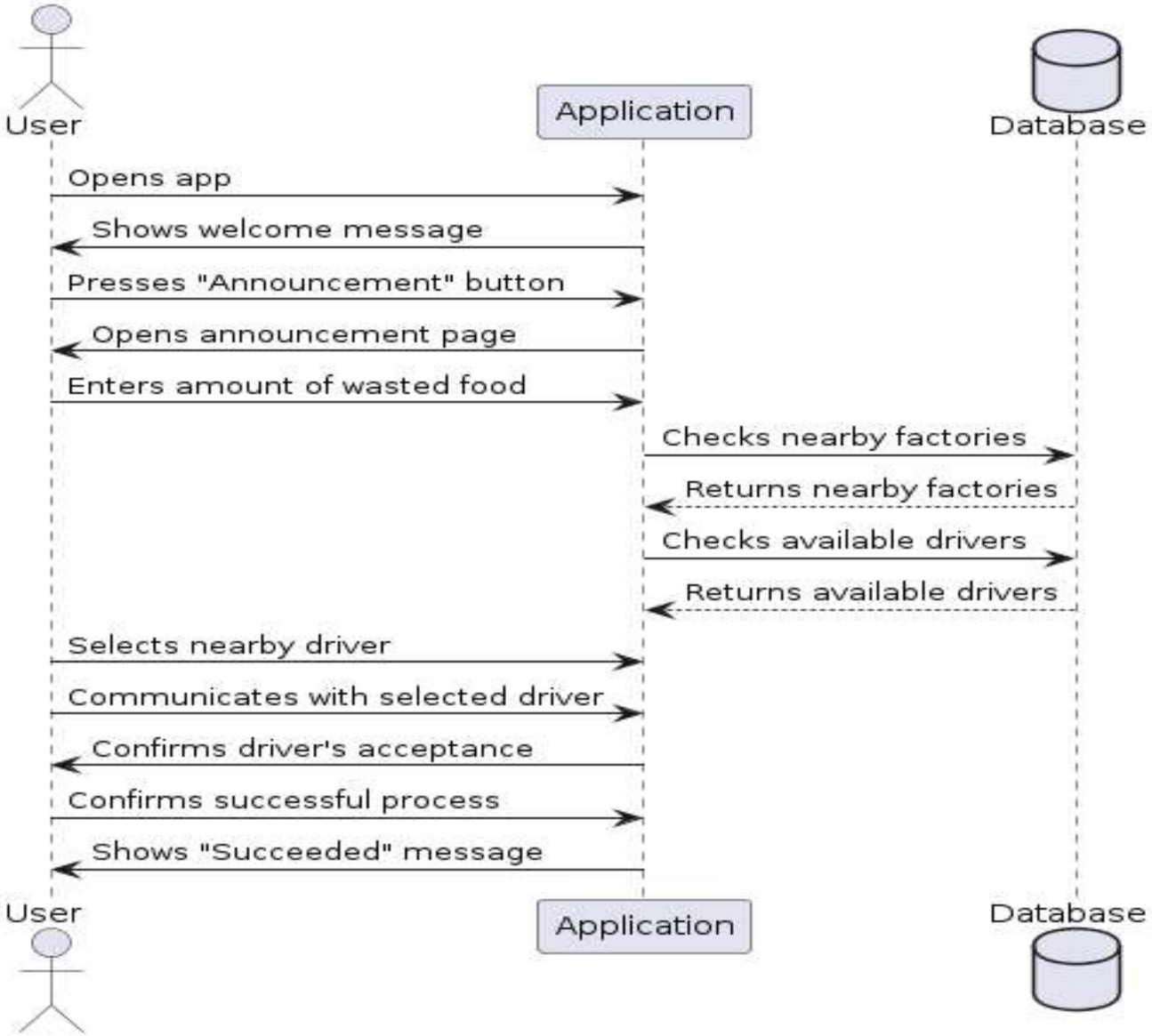
5.5. Sequence diagrams

In order to provide a clear visualization of the interactions within our system, sequence diagrams have been employed as a vital tool in this documentation. Sequence diagrams depict the chronological flow of messages exchanged between various components of the system, illustrating how different parts collaborate to achieve specific functionalities. These diagrams are instrumental in understanding the dynamic behavior of our system, showcasing the sequence of method calls, the order of execution, and the interdependencies among objects and processes. Utilizing PlantUML, an intuitive and versatile tool for diagramming, we have created these sequence diagrams to elucidate the core processes and interactions fundamental to our project's architecture.

5.5.1. Sign Up Sequence Diagram

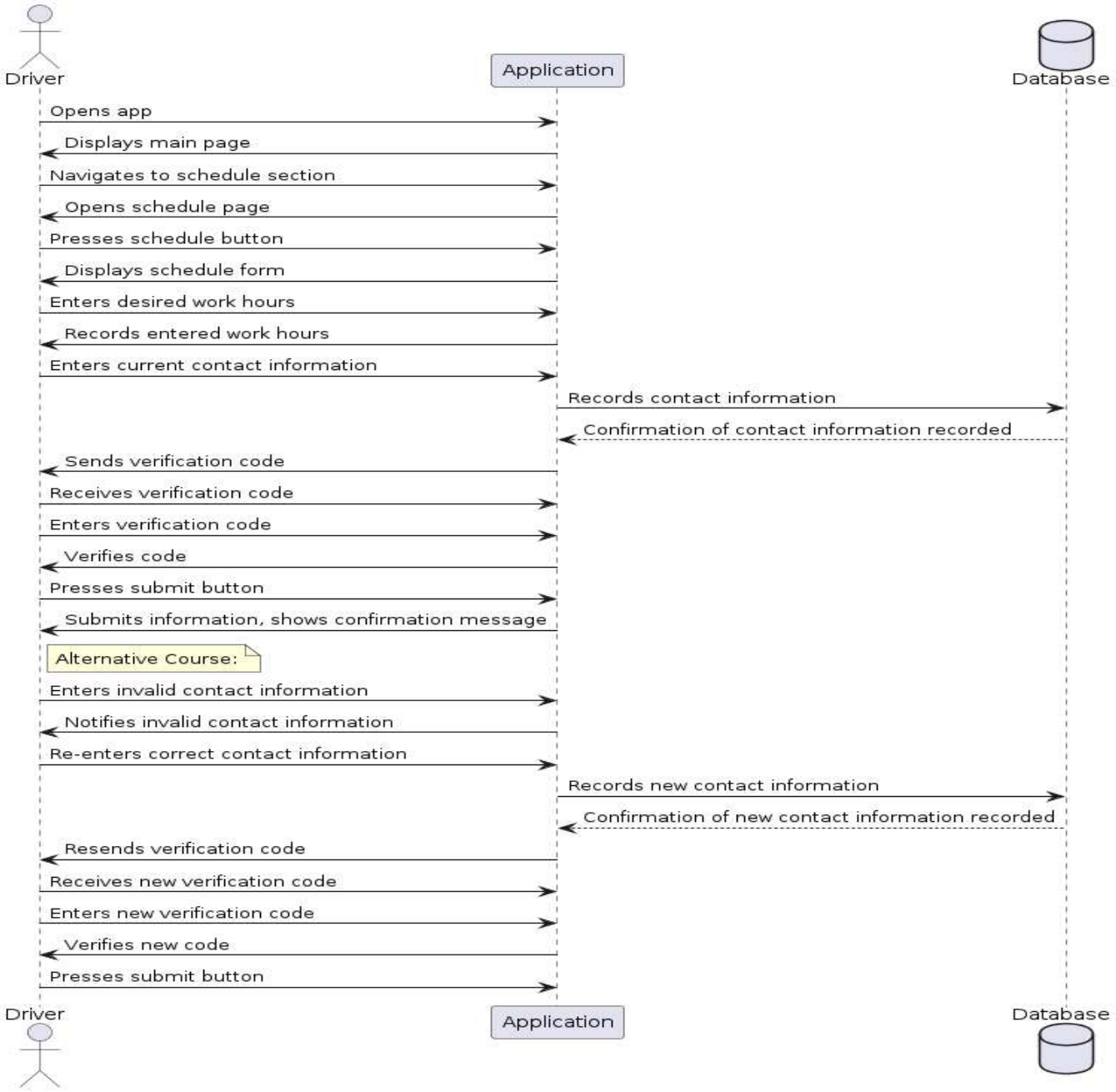
The sequence diagram illustrates the process where a new user (actor) signs up for the application. Initiated by pressing the "Sign up" button, the user provides personal details such as name, birth date, gender, phone number, and email. They also choose their role (restaurant manager, factory manager, or driver). After entering necessary licenses and creating a strong password, the user confirms their information and creates an account. The application verifies data and stores it in the database, ensuring a smooth registration process with error handling for invalid inputs or connectivity issues.

5.5.2. Restaurant Announcement Sequence Diagram



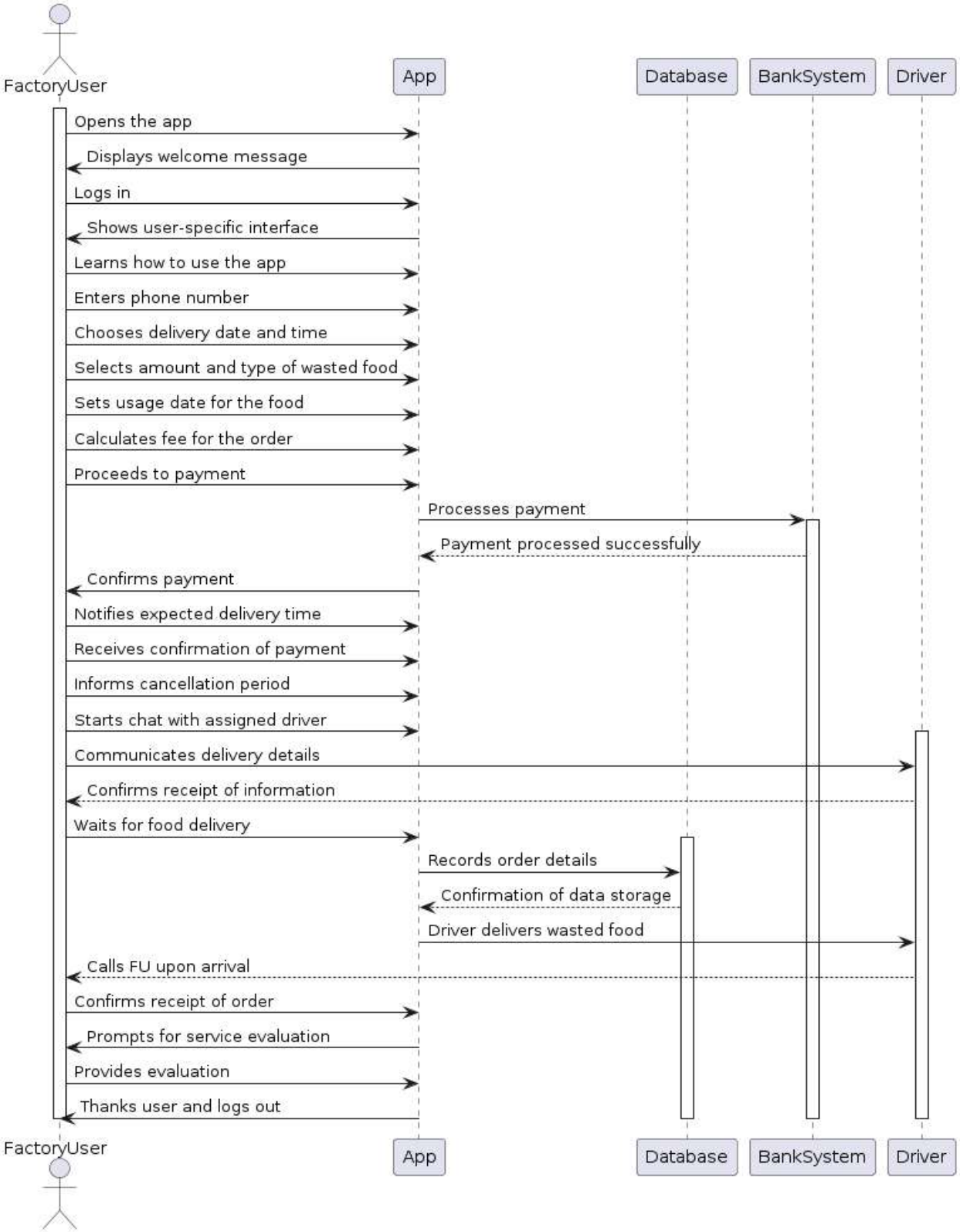
This sequence diagram depicts how restaurant managers announce available wasted food for delivery to nearby factories using the application. Starting from the main page, the manager accesses the announcement feature and enters the quantity of wasted food. The app identifies nearby factories and available drivers, facilitating communication between the manager and chosen driver. Successful completion involves confirming the food transfer, with alternatives for server errors or incorrect food quantities handled. Post-conditions ensure the manager successfully gets rid of excess food while drivers utilize their availability efficiently.

5.5.3. Driver Schedule Sequence diagram



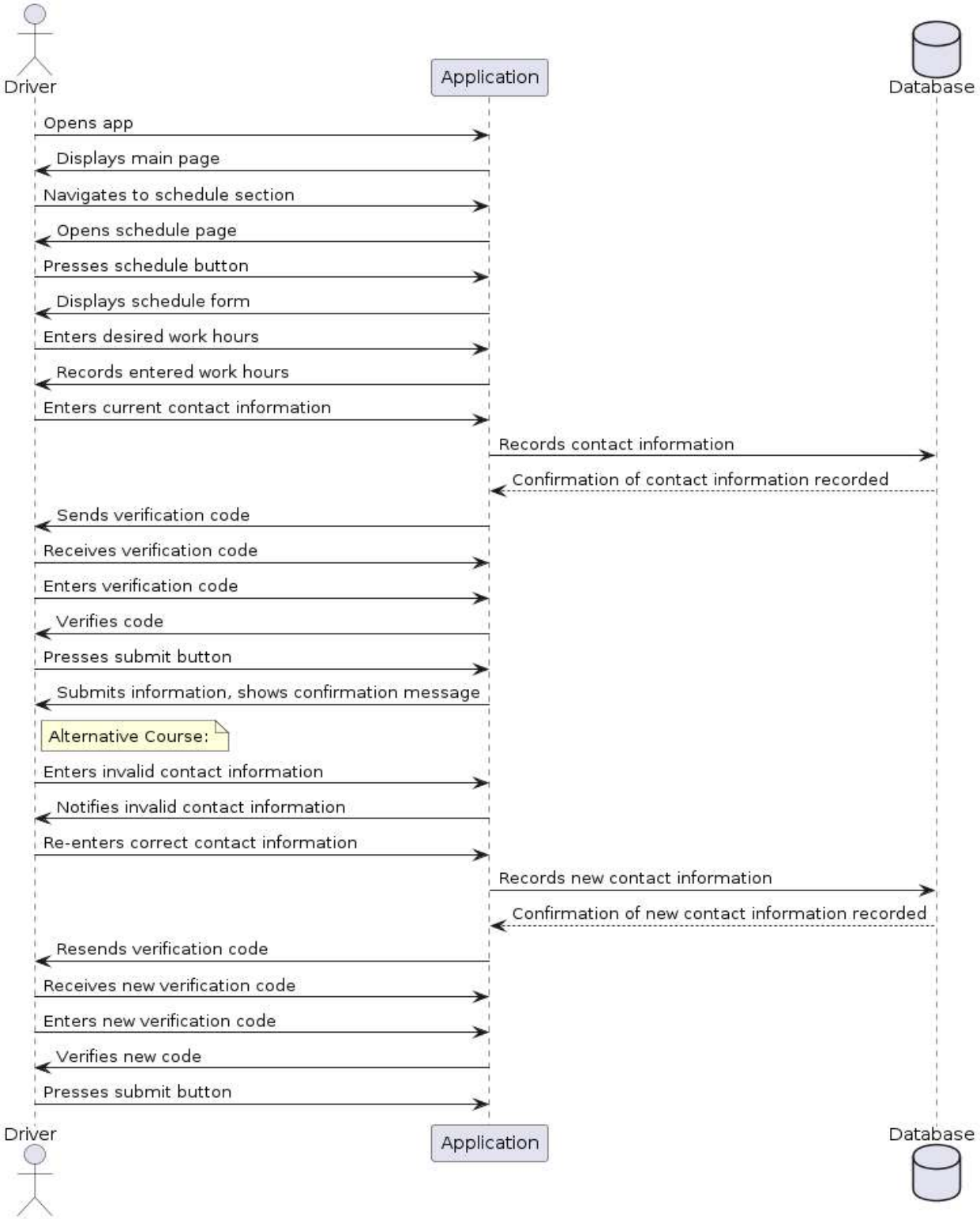
Focused on drivers, this diagram outlines how they announce their available work hours within the application. Starting from the main page, the driver navigates to the schedule section, enters their preferred work hours, and updates their contact information. The application sends a verification code to validate the contact information, ensuring accuracy. Alternatives include handling invalid contact details, prompting re-entry until successful. Successful submission confirms availability, allowing drivers to view and accept delivery opportunities, benefiting from efficient scheduling and resource utilization.

5.5.4. Factory User Sequence Diagram



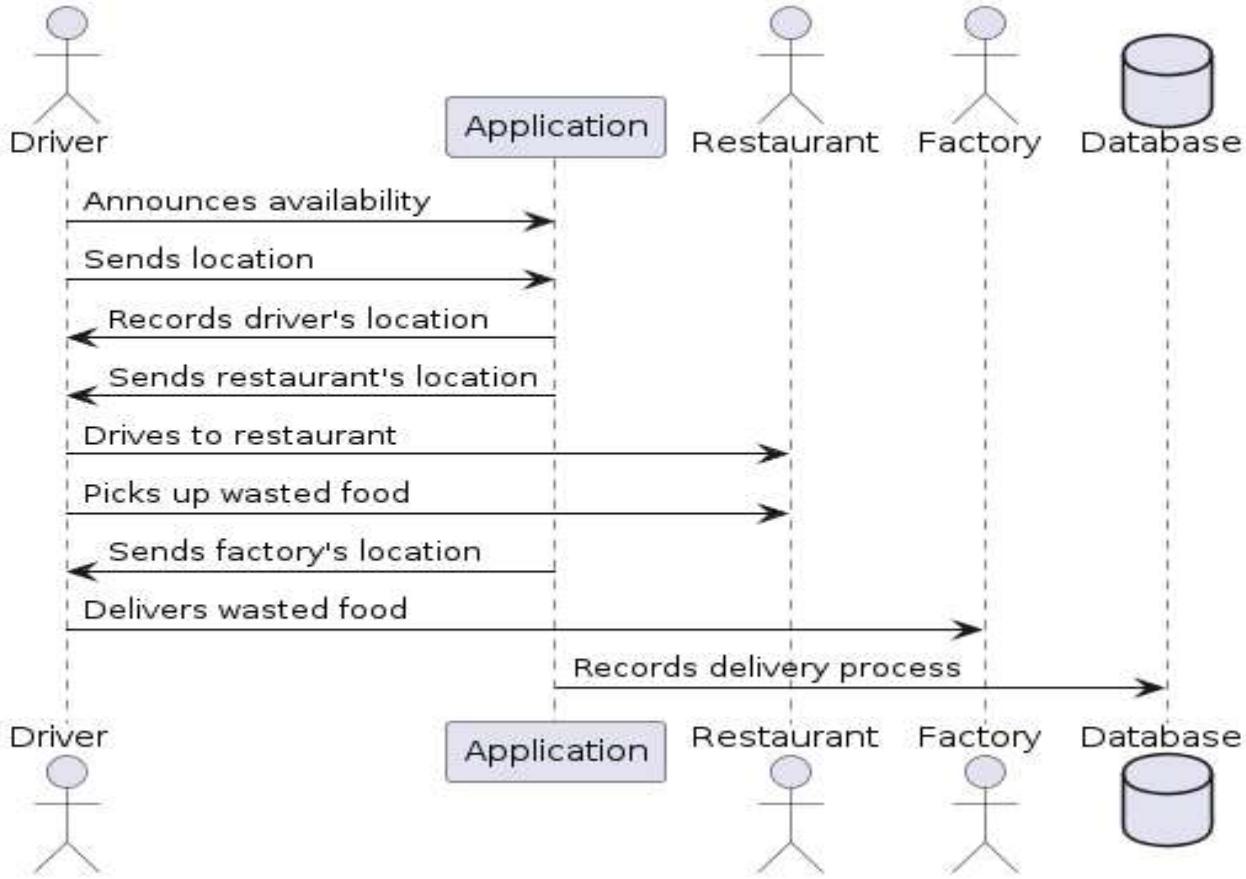
The sequence diagram for the "Factory User" use case in the "Food for Good" application outlines a systematic interaction flow between the Factory User, the application, payment processing system, and the Driver. Initially, the Factory User logs into the app and navigates to request wasted food from nearby restaurants, specifying the quantity and type needed along with the desired usage date. After selecting the food, the user proceeds to make payment via a bank card, facilitated through integration with the Bank System. Upon successful payment, the app confirms the transaction, provides an estimated delivery time, and informs the user about the cancellation policy. The Factory User then communicates with the assigned Driver through the app to coordinate the food pickup and delivery to their factory. Once delivered, the Driver contacts the Factory User for confirmation, which is acknowledged through the app. Finally, the Factory User evaluates the service received and logs out from the application, ensuring a structured and efficient process for managing food waste delivery from restaurants to factories.

5.5.5. Chatting Sequence Diagram



This diagram illustrates communication among drivers, restaurant managers, and factory managers through the application to coordinate wasted food transportation. The factory manager initiates by searching for restaurants with suitable food quantities, prompting the restaurant manager to contact an available driver. Communication continues until the driver successfully picks up and delivers the food. Alternatives involve managing communication errors like internet disconnections, ensuring successful food transfers despite delays. Post-conditions include comprehensive communication records, benefiting all parties involved in efficient food redistribution.

5.5.6. Driver Delivery Sequence Diagram



In this diagram, the driver uses the application to pick up wasted food from a restaurant and deliver it to a factory. The driver first announces availability and sends their location to the application, which then directs them to the restaurant. After picking up the food, the app provides directions to the factory for delivery. Alternatives manage scenarios like distant restaurants or insufficient space, redirecting the driver as needed. Post-conditions confirm successful delivery, ensuring payment for the driver and restaurant while maintaining comprehensive delivery records in the database.

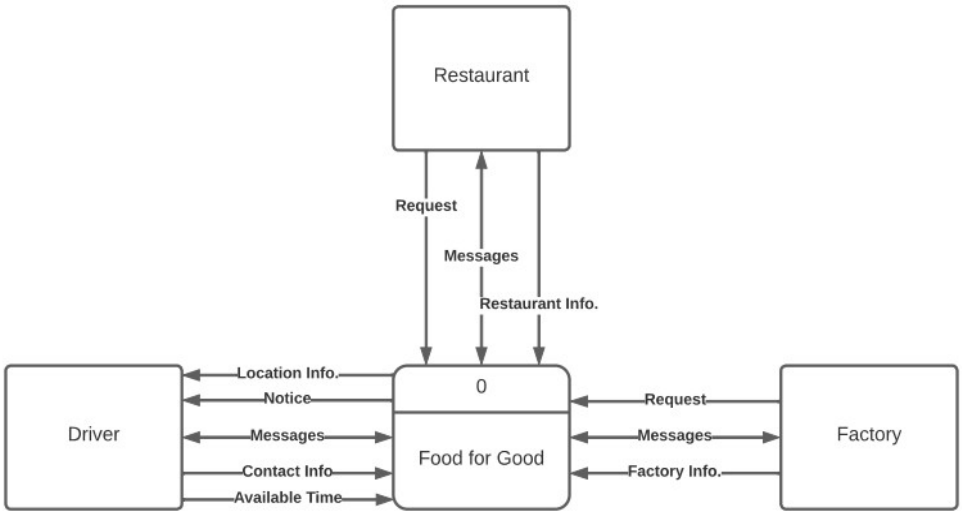
5.6. Data Flow Diagrams

5.6.1. Context Diagram

The context diagram (DFD Level 0) for the "Food for Good" application provides a high-level overview of the system's interaction with external entities. It illustrates the flow of information between the main system and its stakeholders: restaurants, drivers, and organic fertilizer factories. The primary interactions include:

- Restaurants sending food waste details (Request and Restaurant Info) to the system and receiving updates (Messages).
- Drivers providing their location, availability, and contact information to the system and receiving notices and messages.
- Factories communicating their requirements and receiving notifications about food waste deliveries.

Ultimately, this diagram sets the foundational understanding of how the system will interact with its primary users and external entities.



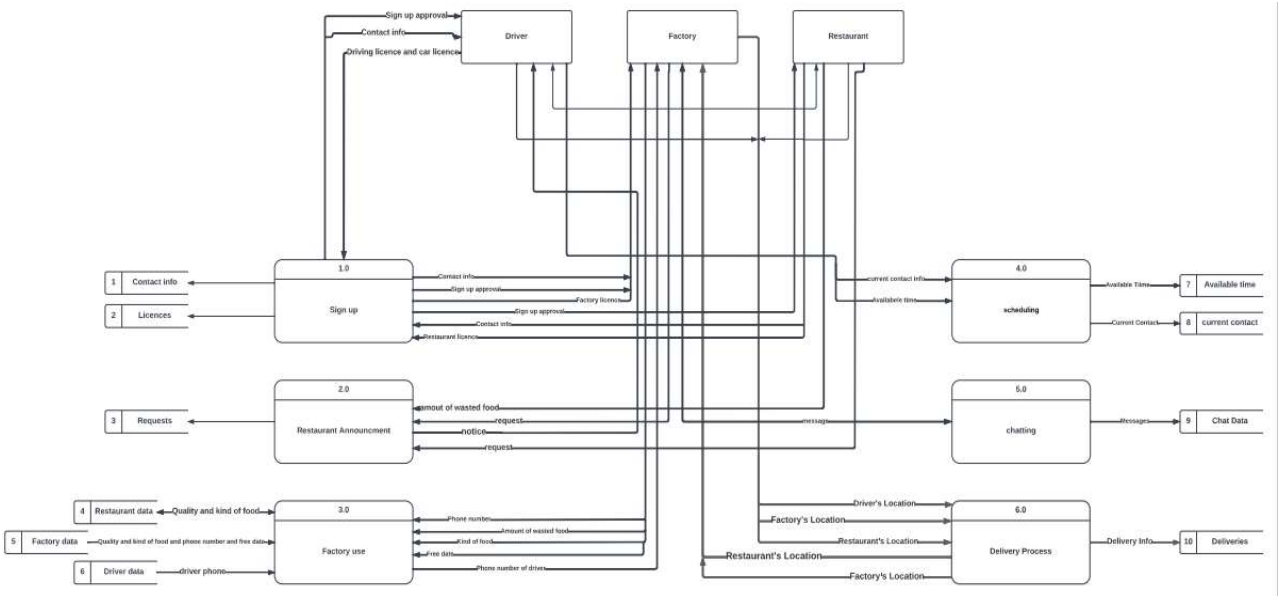
5.6.2. DFD 1: System Overview

The system overview (DFD Level 1) expands on the context diagram by breaking down the "Food for Good" application into its main processes. The key processes include:

- **Sign Up (1.0):** Handles the registration and approval of users (restaurants, drivers, and factories). Users provide contact information and licenses, which are verified and approved by the admin.

- **Restaurant Announcement (2.0):** Allows restaurants to announce the amount of food waste available for collection.
- **Factory Use (3.0):** Manages the data from factories regarding the type and amount of food waste they can process.
- **Scheduling (4.0):** Coordinates the availability and scheduling of drivers for food waste pickup and delivery.
- **Chatting (5.0):** Facilitates communication between restaurants, drivers, and factories.
- **Delivery Process (6.0):** Manages the logistics of picking up food waste from restaurants and delivering it to factories.

This diagram provides a more detailed view of the system's internal processes and their interactions.



5.6.3. DFD 2: Sign Up Process

The detailed sign-up process (DFD Level 2) illustrates the steps involved in registering new users (restaurants, drivers, and factories) for the "Food for Good" application. The steps include:

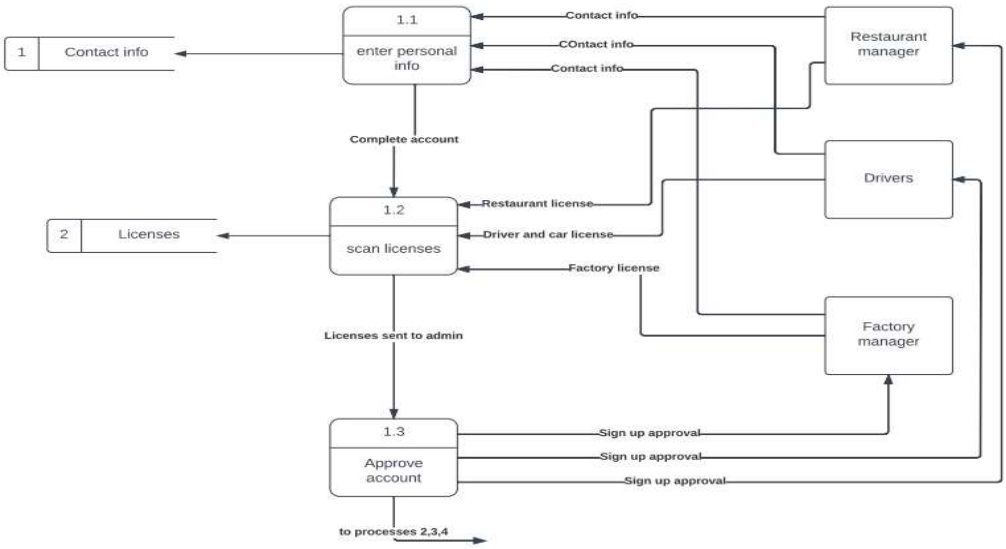
- **Enter Personal Info (1.1):** Users input their contact information to create an account.
- **Scan Licenses (1.2):** Users scan and upload the necessary licenses (restaurant, driver, and factory licenses) for verification.
- **Approve Account (1.3):** The admin reviews the submitted information and licenses, then approves or rejects the account.

This detailed diagram ensures that all necessary information is collected and verified before users can access the system, maintaining the integrity and trustworthiness of the platform.

5.6.4. DFD 4: Request Management and Factory Interaction

The DFD 4 focuses on managing requests from restaurants and coordinating with factories to determine suitable destinations for the wasted food. The primary interactions include:

- **Restaurant to System:** Restaurants send information about the amount of wasted food and initiate a request.
- **System to Factory:** The system determines a suitable factory for processing the food waste and sends the factory information to the system.
- **System to Driver:** The system finds and notifies a driver about the request.
- **System to Requests:** The system confirms the request and updates the list of active requests.

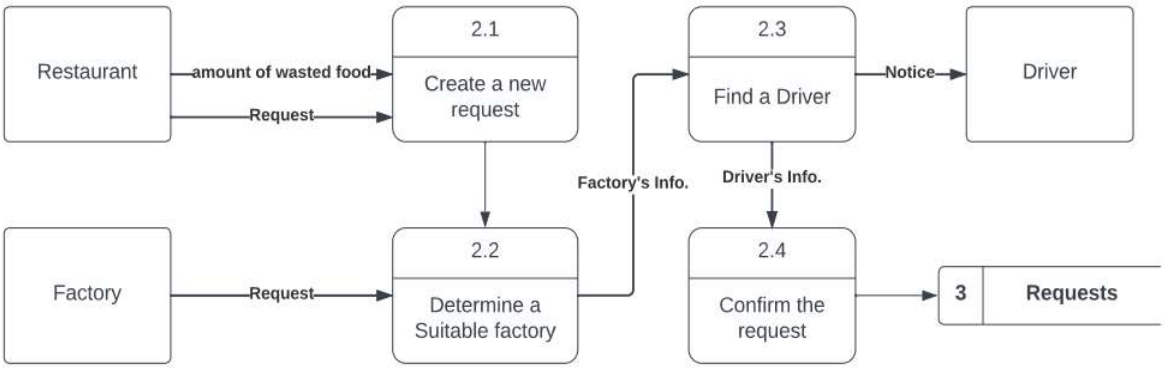


5.6.5. DFD 5: Factory Manager Operations

The DFD 5 outlines the operations managed by the factory manager, including communication with the system, restaurants, and drivers. The primary interactions include:

- **System to Factory Manager:** The system confirms the factory manager's phone number and provides necessary details for coordination.

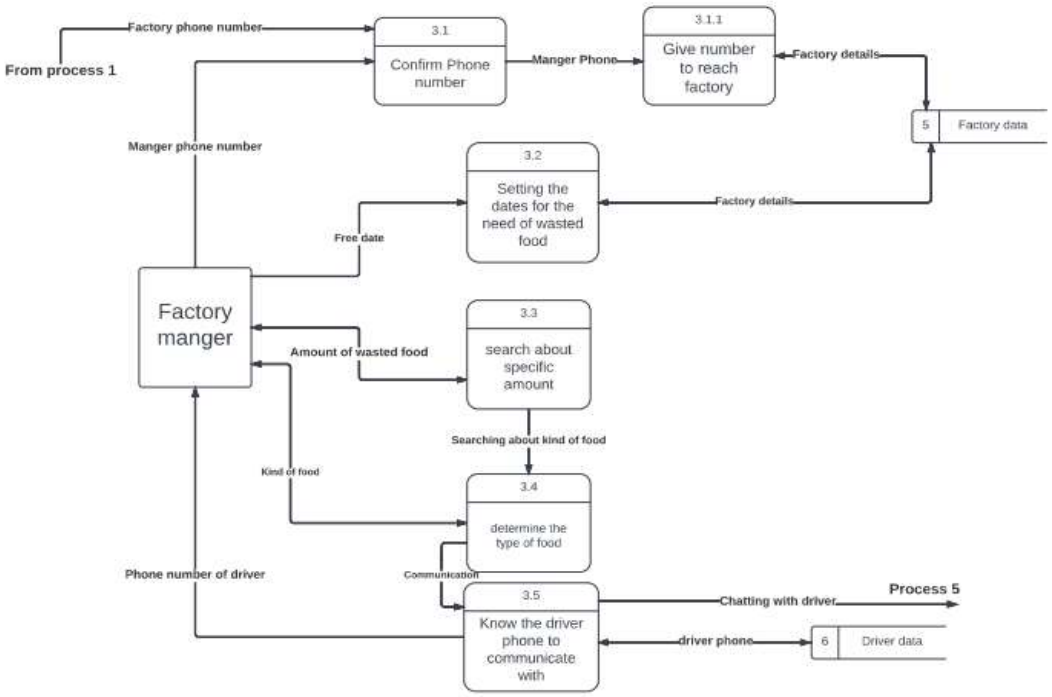
- **Factory Manager to System:** The factory manager sets the dates for the food waste needs, searches for specific amounts and types of food waste, and determines suitable food types for processing.
- **Factory Manager to Driver:** The factory manager communicates with the driver to coordinate the delivery of food waste.



5.6.6. DFD 6: Matching Restaurant Needs with Factory Requirements

The DFD 6 details the process of matching the quantity and types of food waste from restaurants with the needs of factories. The primary interactions include:

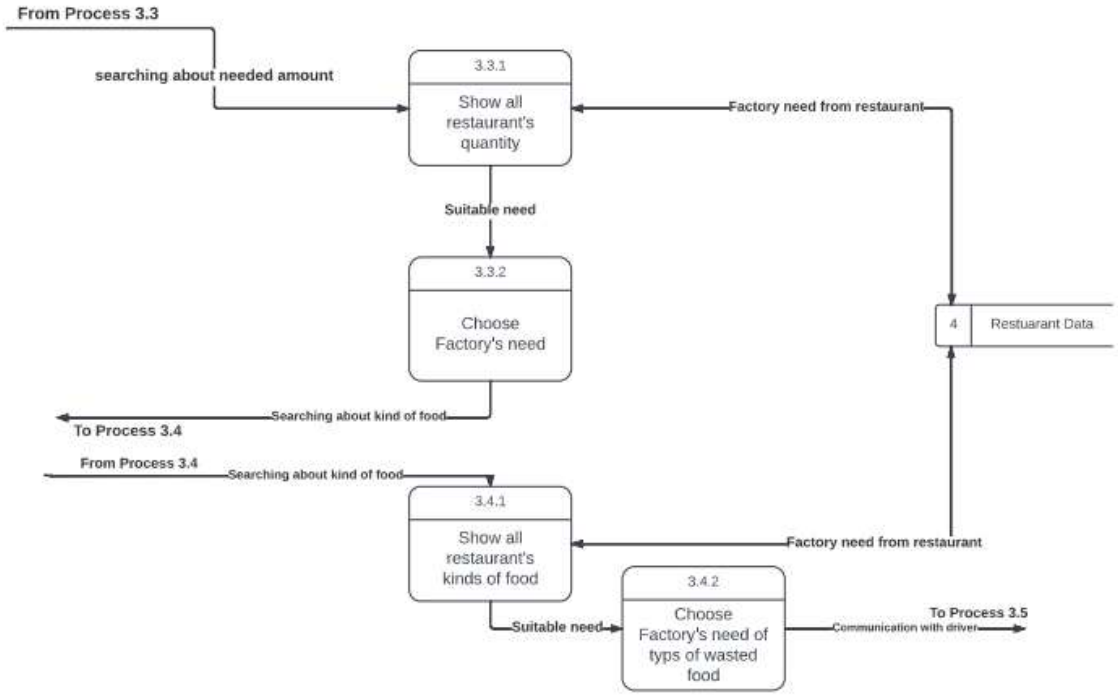
- **System to Factory Manager:** The factory manager searches for and selects the required quantities and types of food waste from restaurants.
- **System to Restaurant Data:** The system provides data on all restaurants' quantities and kinds of food waste to help the factory manager determine suitable matches.
- **Factory Manager to Driver:** Once the factory's needs are matched with available food waste, the factory manager communicates with the driver for pickup and delivery.



5.6.7. DFD 7: Factory Needs and Selection Process

The DFD Level 2 for Process 3.3 and 3.4 (Factory Needs and Selection Process) elaborates on how the "Food for Good" application manages the selection of food waste quantities and types from restaurants to meet the needs of organic fertilizer factories.

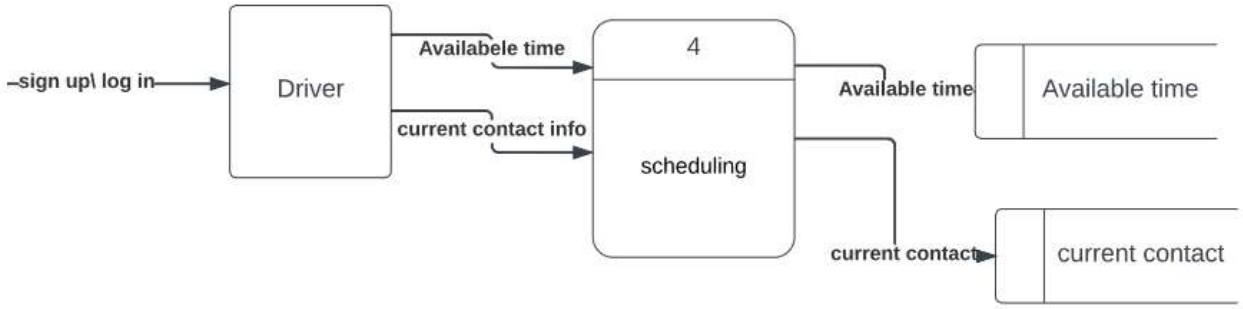
- **Process 3.3: Choosing Food Quantities**
  - Show all restaurant's quantity (3.3.1) involves displaying the total quantities of food waste available from various restaurants based on the specific needs of the factories. The input for this process is the factory needs from restaurants, and the output is a display of all available food waste quantities.
  - Choose Factory's need (3.3.2) is the process where the system selects and matches the required food waste quantities to meet the factory's needs. The input is suitable food waste quantities, and the output is the selected quantities that match factory requirements, ensuring the correct amount is chosen for processing.
- **Process 3.4: Choosing Food Types**
  - Show all restaurant's kinds of food (3.4.1) involves showing the different types of food waste available from various restaurants to meet the specific needs of the factories. The input for this process is the factory needs from restaurants, and the output is a display of all available types of food waste.



5.6.8. DFD 8: Driver Scheduling

The DFD Level 2 for Process 4 (Driver Scheduling) details how the "Food for Good" application coordinates the scheduling of drivers for food waste collection and delivery.

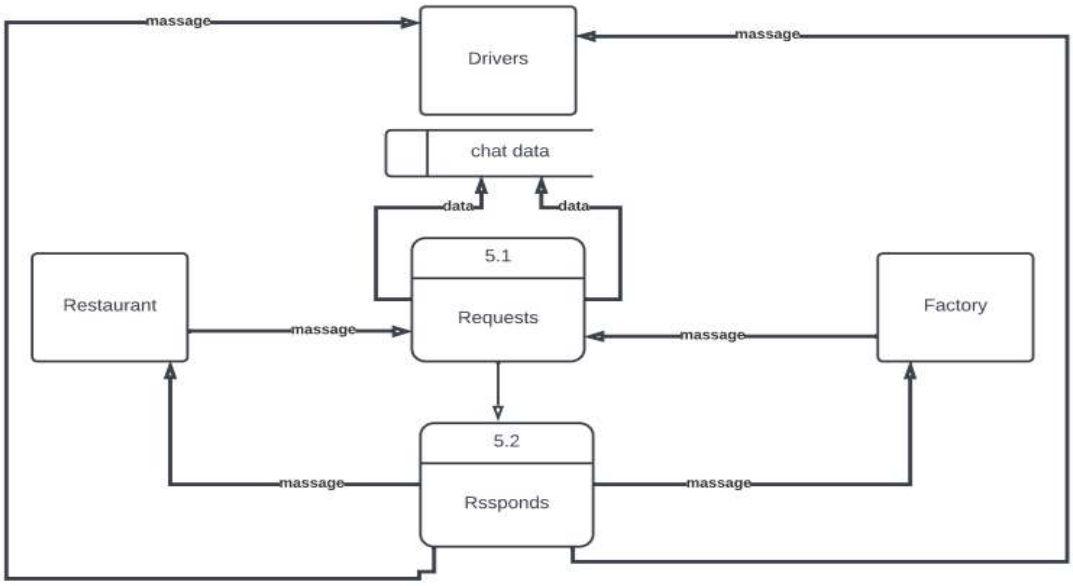
- **Driver**:
  - **Input**: Sign-up/log-in details, current contact information, available time.
  - **Output**: Driver's availability and contact information.
  - **Description**: Drivers log into the system and provide their availability and contact details, which are used for scheduling purposes.
- **Scheduling**:
  - **Input**: Driver's available time and contact information.
  - **Output**: Scheduled pickups and deliveries.
  - **Description**: The system coordinates the availability and schedules of drivers to ensure efficient collection and delivery of food waste from restaurants to factories.



5.6.9. DFD 9: Communication Management

The DFD Level 2 for Process 5 (Communication Management) illustrates how the "Food for Good" application facilitates communication between restaurants, drivers, and factories to ensure efficient coordination of food waste collection and delivery.

- **Requests (5.1)**:
  - **Input**: Messages from restaurants and factories.
  - **Output**: Processed requests for food waste collection and delivery.
  - **Description**: This process involves gathering and processing requests from restaurants and factories, ensuring that the necessary information is available for coordination.
- **Responds (5.2)**:
  - **Input**: Processed request data.
  - **Output**: Response messages to restaurants and factories.
  - **Description**: The system sends response messages to restaurants and factories, informing them about the status and details of their requests.

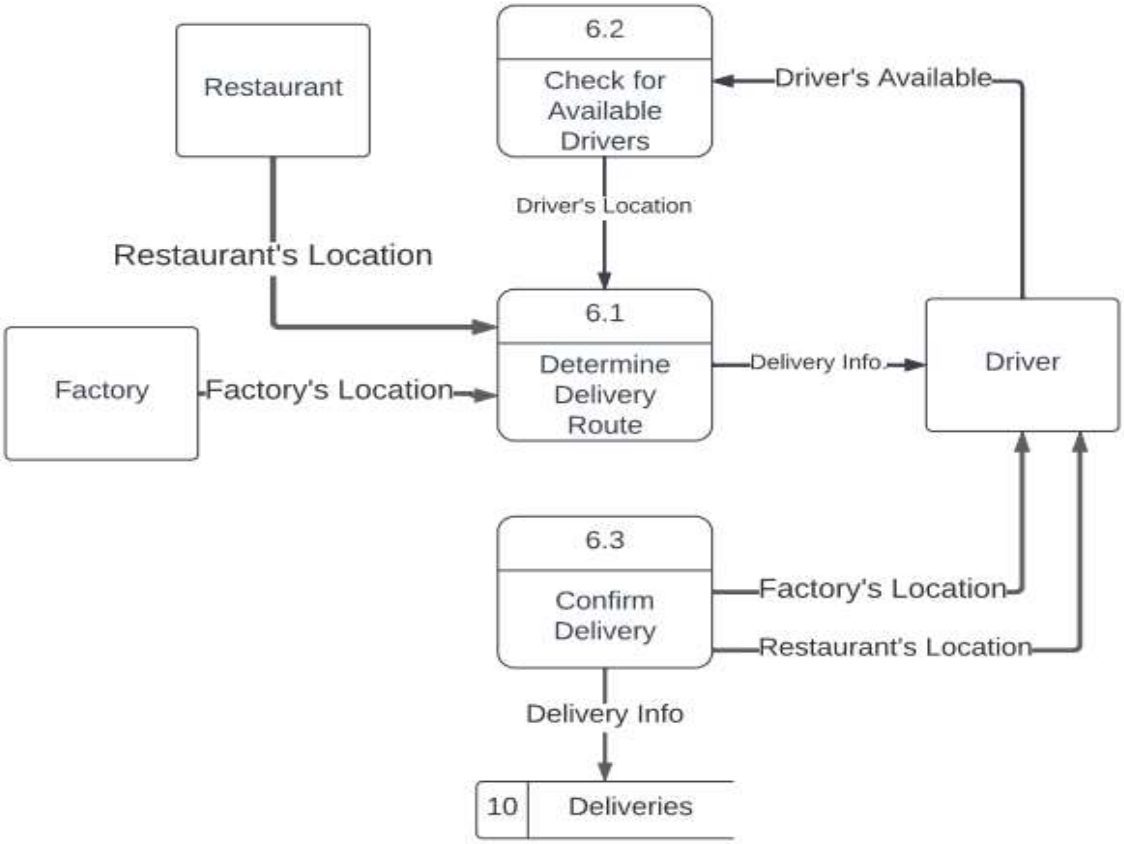


5.6.10. DFD 10: Final Delivery Process Workflow

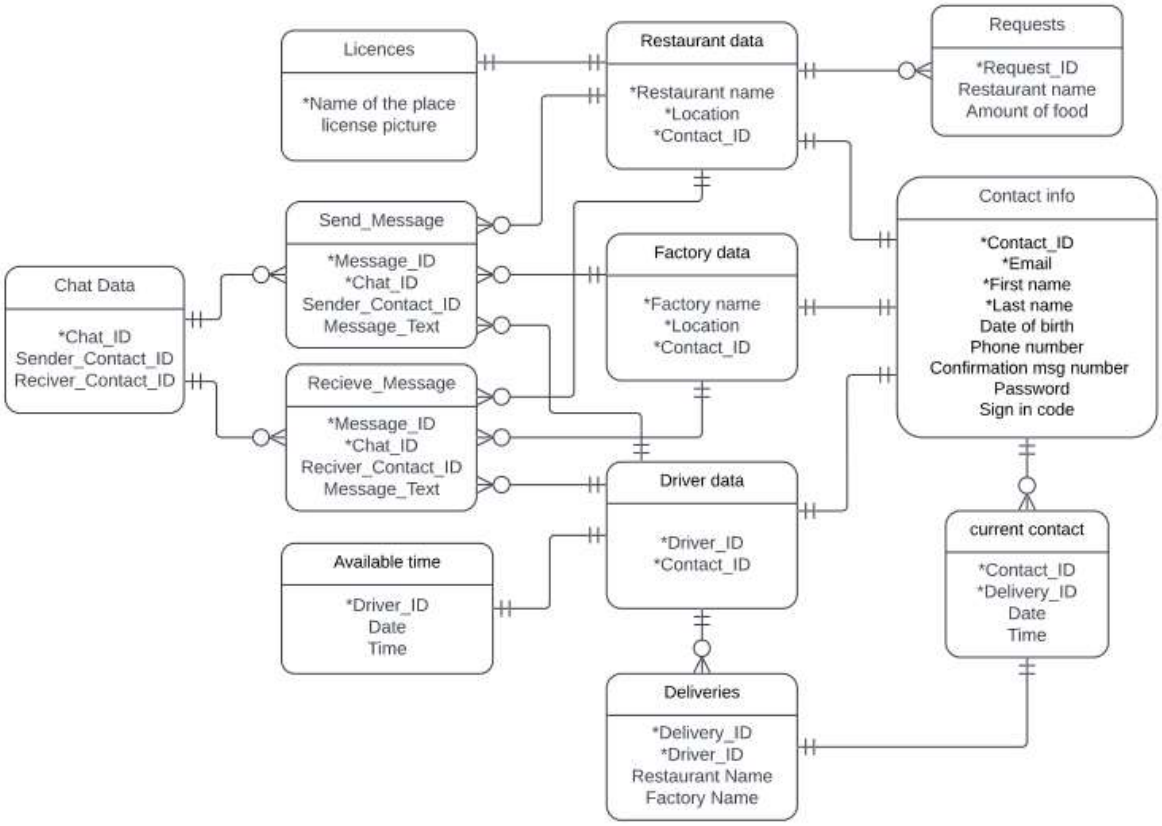
The "Final Delivery Process Workflow" (DFD Level 10) for the "Food for Good" application encapsulates the detailed steps involved in the process of delivering food waste from restaurants to organic fertilizer factories. This diagram elaborates on the specific interactions and sequence of events among the main system, restaurants, drivers, and factories:

- **Restaurant and Factory Locations:** The process initiates with the system receiving location data for both the restaurant and the factory involved in the transaction.
- **Checking for Available Drivers:** Using the driver's location information, the system identifies available drivers in proximity to the restaurant.
- **Determining the Delivery Route:** Once a suitable driver is available, the system calculates the optimal route for the delivery from the restaurant to the factory. This step ensures efficient logistics management, minimizing travel time and environmental impact.
- **Confirmation of Delivery:** The driver confirms the successful collection of food waste from the restaurant and its subsequent delivery to the factory. This step involves the transmission of delivery information back to the system, which updates both the restaurant and the factory about the status of the delivery.

- **Recording Deliveries:** The final step involves logging the details of the delivery into the system. This provides a record of transactions for future reference and accountability.


Restaurant
6.2
Check for Available Drivers
Driver's Available
Driver's Location
Restaurant's Location
6.1
Determine Delivery Route
Delivery Info
Driver
Factory
Factory's Location
6.3
Confirm Delivery
Factory's Location
Restaurant's Location
Delivery Info
10
Deliveries

5.7. ER Diagram



This Entity-Relationship Diagram (ERD) outlines the data structure for a multi-faceted delivery management system designed to facilitate communication and operations between various entities including restaurants, factories, and drivers. The system manages several key entities such as Licenses, Restaurant Data, Factory Data, Deliveries, and Driver Data, each linked to relevant Contact Information to ensure efficient coordination. Communication is managed through entities like Chat Data, Send Message, and Receive Message, enabling real-time interaction between users. The 'Requests' entity handles specific delivery requests from restaurants, linking to the amount and type of food required. Driver schedules are managed under 'Available time,' ensuring timely deliveries. This ERD serves as the backbone for the database, ensuring that data flow is logical, consistent, and robust, catering to all needs of a dynamic delivery service.

6. Implementation

6.1. Overview

The implementation phase of the "Food for Good" application focused on developing the mobile application frontend using Flutter. The implementation process followed the Agile methodology, with the development team working in iterative sprints to incrementally build and refine the application.

The application is built using the Model-View-Controller (MVC) architecture, which separates the application logic into three interconnected components:

- **Model**: Manages the data and business logic of the application. This includes defining data models and handling data storage and retrieval (though this is limited to frontend storage like local state management for now).
- **View**: Represents the UI components and handles the presentation logic. It is responsible for displaying data to the user and capturing user input.
- **Controller**: Acts as an intermediary between the Model and the View. It processes user input, updates the Model, and updates the View with the new data.

Using MVC architecture helps in organizing the codebase, making it easier to manage and scale the application. Each component has a specific responsibility, which enhances the maintainability and testability of the application.

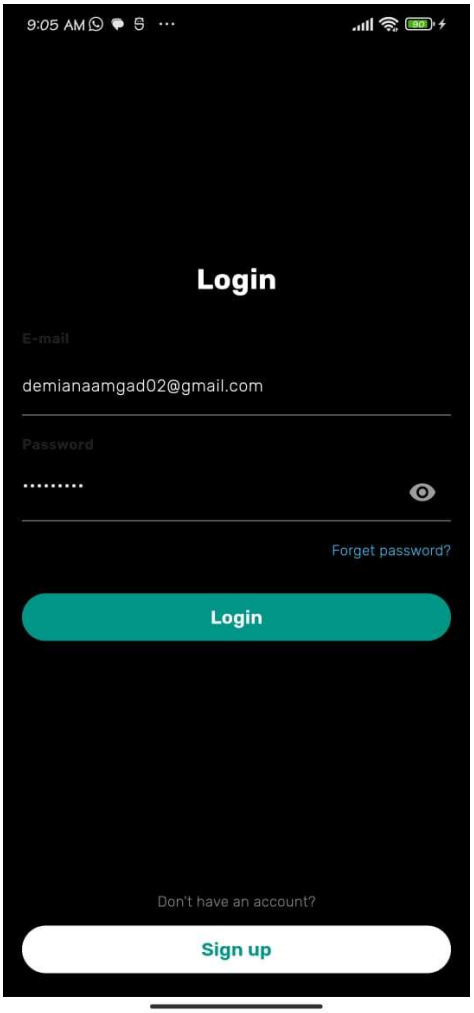
6.2. Code Structure

The codebase is organized into several key directories, each responsible for different aspects of the application:

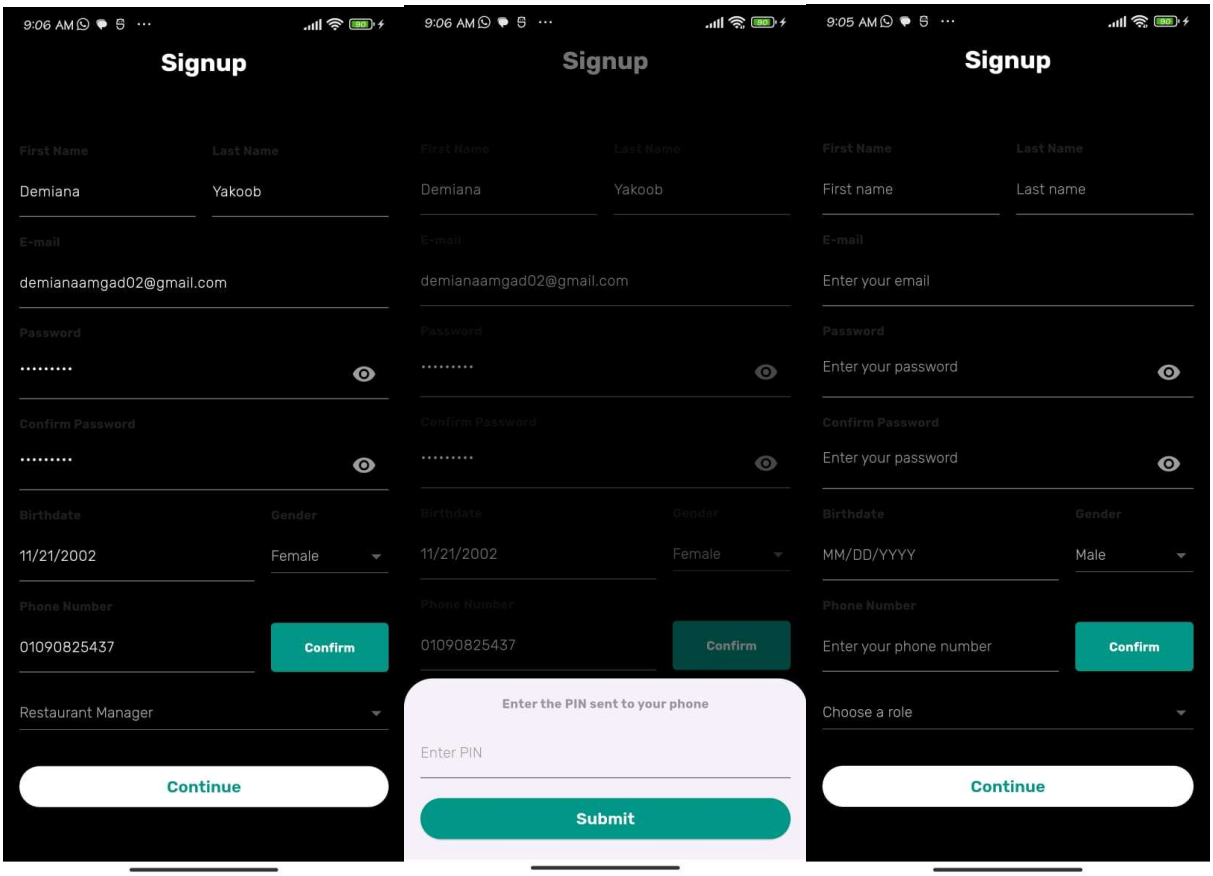
1. **lib/**: Contains the main Flutter application code.
   - **models/**: Defines the data models used throughout the application.
   - **views/**: Contains the different screens (UI components) of the application.
   - **controllers/**: Includes service classes that handle business logic and state management.
   - **views/widgets/**: Custom widgets used across multiple screens.
   - **views/screens/**: Different frontend screens used in the application

6.3. Screenshots

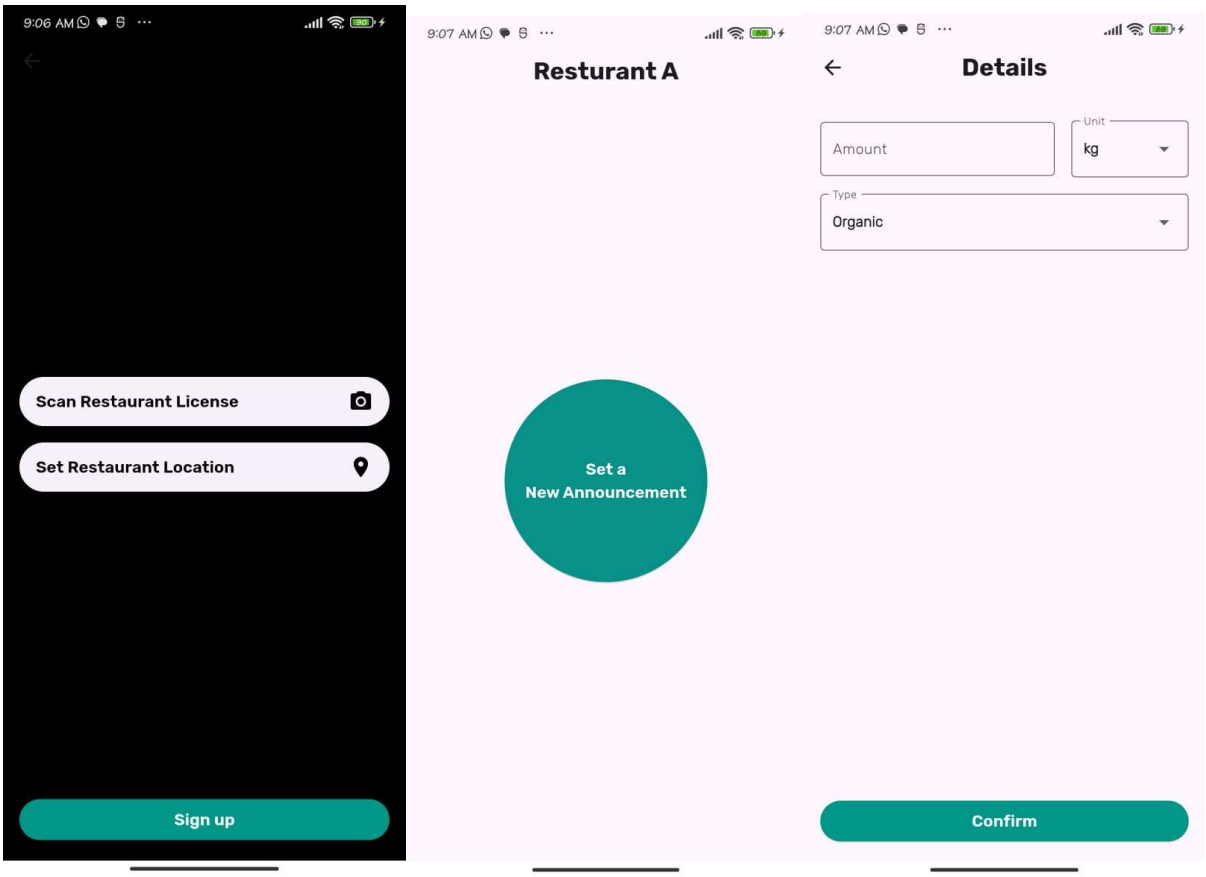
Login



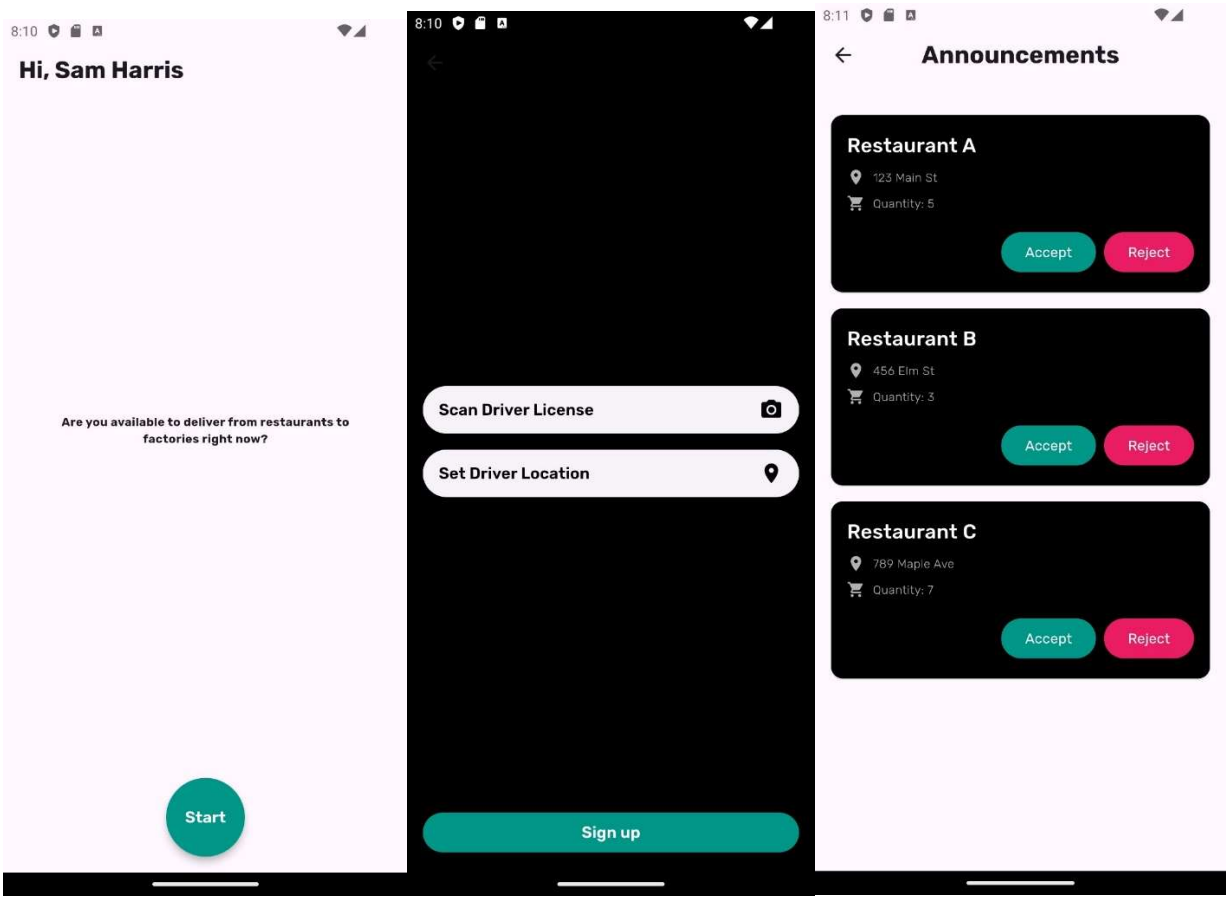
Sign up


9:06 AM
Signup
First Name
Last Name
Demiana
Yakoob
E-mail
demianaamgad02@gmail.com
Password
Confirm Password
Birthdate
Gender
11/21/2002
Female
Phone Number
01090825437
Confirm
Restaurant Manager
Continue
9:06 AM
Signup
Demiana
Yakoob
demianaamgad02@gmail.com
11/21/2002
Female
01090825437
Confirm
Enter the PIN sent to your phone
Enter PIN
Submit
9:05 AM
Signup
First Name
Last Name
First name
Last name
E-mail
Enter your email
Password
Enter your password
Confirm Password
Enter your password
Birthdate
Gender
MM/DD/YYYY
Male
Phone Number
Enter your phone number
Confirm
Choose a role
Continue

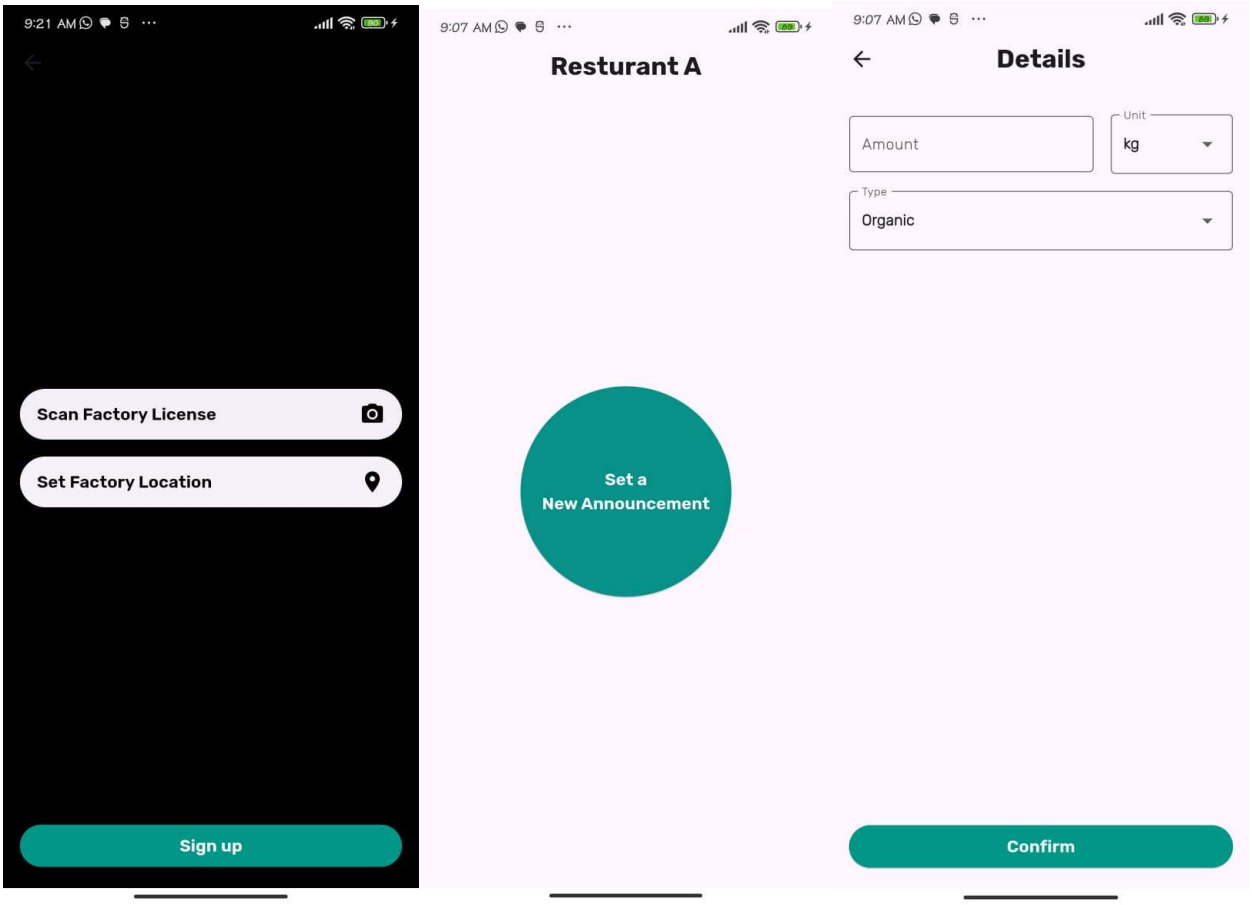
Restaurant Manager



Driver


8:10
Hi, Sam Harris
Are you available to deliver from restaurants to factories right now?
Start
8:10
Scan Driver License
Set Driver Location
Sign up
8:11
Announcements
Restaurant A
123 Main St
Quantity: 5
Accept
Reject
Restaurant B
456 Elm St
Quantity: 3
Accept
Reject
Restaurant C
789 Maple Ave
Quantity: 7
Accept
Reject

Factory Manager



7. Testing

7.1. Testing Strategies

The testing strategies employed for the "Food for Good" application include:

- **Unit Testing**: Focused on testing individual components and functions to ensure they work as intended in isolation.
- **Integration Testing**: Verified that different parts of the application work together seamlessly.
- **UI Testing**: Ensured that the user interface elements function correctly and provide a good user experience across various devices and screen sizes.
- **End-to-End (E2E) Testing**:
    - Simulates real user scenarios to test the entire application flow from start to finish.

- Example: Testing a complete flow from logging in, creating a food waste listing, and tracking delivery.

7.2. Test Cases

7.2.1. Test Case: Validate Email Field

Description: Verify that the email validation works correctly.

Steps:

1. Launch the application.
2. Enter an invalid email in the email field (e.g., "invalidEmail").
3. Enter a valid email in the email field (e.g., "test@example.com").
4. Leave the email field empty.

Expected Results:

- Step 2: An error message "Enter a valid email" should be displayed.
- Step 3: No error message should be displayed.
- Step 4: An error message "Email cannot be empty" should be displayed.

Actual Results:

- Entering an invalid email displays "Enter a valid email".
- Entering a valid email displays no error message.
- Leaving the email field empty displays "Email cannot be empty".

Comments: Email validation works as expected.

7.2.2. Test Case: Validate Password Field

Description: Verify that the password validation works correctly.

Steps:

1. Launch the application.
2. Enter a password shorter than 8 characters (e.g., "short").
3. Enter a valid password with 8 or more characters (e.g., "validPassword").
4. Leave the password field empty.

Expected Results:

- Step 2: An error message "Password must be at least 8 characters" should be displayed.
- Step 3: No error message should be displayed.
- Step 4: An error message "Password cannot be empty" should be displayed.

Actual Results:

- Entering a short password displays "Password must be at least 8 characters".
- Entering a valid password displays no error message.
- Leaving the password field empty displays "Password cannot be empty".

Comments: Password validation works as expected.

7.2.3. Test Case: Validate Login Button Functionality

Description: Ensure that the "Login" button only works when both email and password are valid.

Steps:

1. Launch the application.
2. Enter a valid email and password.
3. Press the "Login" button.
4. Enter an invalid email and/or password.
5. Press the "Login" button.

Expected Results:

- Step 2-3: If both fields are valid, the `onLoginPressed` callback should be triggered.
- Step 4-5: If any field is invalid, the `onLoginPressed` callback should not be triggered.

Actual Results:

- With valid email and password, `onLoginPressed` is triggered.
- With invalid email and/or password, `onLoginPressed` is not triggered.

Comments: "Login" button functionality works as expected.

7.2.4. Test Case: Validate "Forget Password" Link

Description: Ensure that the "Forget password?" link works as expected.

Steps:

1. Launch the application.

2. Press the "Forget password?" link.

Expected Results:

- Step 2: The "Forget password?" logic should be executed.

Actual Results:

- Pressing the "Forget password?" link executes the logic.

Comments: The "Forget password?" link works as expected.

7.2.5. Test Case: Validate First Name Field

Description: Verify that the first name validation in the signup form works correctly.

Steps:

1. Launch the application.
2. Navigate to the signup form.
3. Leave the first name field empty.

Expected Results:

- An error message "Name cannot be empty" should be displayed.

Actual Results:

- Leaving the first name field empty displays "Name cannot be empty".

Comments: First name validation works as expected.

7.2.6. Test Case: Validate Password Field

Description: Verify that the password validation in the signup form works correctly.

Steps:

1. Launch the application.
2. Navigate to the signup form.
3. Enter a password shorter than 8 characters (e.g., "short").
4. Enter a valid password with 8 or more characters (e.g., "validPassword").
5. Leave the password field empty.

Expected Results:

- Step 3: An error message "Password must be at least 8 characters" should be displayed.
- Step 4: No error message should be displayed.
- Step 5: An error message "Password cannot be empty" should be displayed.

Actual Results:

- Entering a short password displays "Password must be at least 8 characters".
- Entering a valid password displays no error message.
- Leaving the password field empty displays "Password cannot be empty".

Comments: Password validation works as expected.

7.2.7. Test Case: Validate Confirm Password Field

Description: Verify that the confirm password validation in the signup form works correctly.

Steps:

1. Launch the application.
2. Navigate to the signup form.
3. Enter a password in the password field (e.g., "validPassword").
4. Enter a different password in the confirm password field (e.g., "differentPassword").
5. Enter the same password in both password fields (e.g., "validPassword" and "validPassword").
6. Leave the confirm password field empty.

Expected Results:

- Step 4: An error message "Passwords do not match" should be displayed.
- Step 5: No error message should be displayed.
- Step 6: An error message "Confirm password cannot be empty" should be displayed.

Actual Results:

- Entering different passwords displays "Passwords do not match".
- Entering the same password displays no error message.
- Leaving the confirm password field empty displays "Confirm password cannot be empty".

Comments: Confirm password validation works as expected.

7.2.8. Test Case: Validate Phone Number Field

Description: Verify that the phone number validation in the signup form works correctly.

Steps:

1. Launch the application.
2. Navigate to the signup form.
3. Enter an invalid phone number in the phone field (e.g., "123").
4. Enter a valid phone number in the phone field (e.g., "12345678901").
5. Leave the phone field empty.

Expected Results:

- Step 3: An error message "Enter a valid phone number" should be displayed.
- Step 4: No error message should be displayed.
- Step 5: An error message "Phone number cannot be empty" should be displayed.

Actual Results:

- Entering an invalid phone number displays "Enter a valid phone number".
- Entering a valid phone number displays no error message.
- Leaving the phone field empty displays "Phone number cannot be empty".

Comments: Phone number validation works as expected.

7.2.9. Test Case: Validate Signup Button Functionality

Description: Ensure that the "Continue" button is only enabled when all required fields are valid.

Steps:

1. Launch the application.
2. Navigate to the signup form.
3. Enter valid data in all required fields.
4. Enter invalid data in one or more required fields.
5. Ensure the "Continue" button is only active when all fields are valid.

Expected Results:

- Step 3: The "Continue" button should be active.
- Step 4: The "Continue" button should be inactive.
- Step 5: The "Continue" button should only be active when all fields are valid.

Actual Results:

- With valid data in all fields, the "Continue" button is active.
- With invalid data in any field, the "Continue" button is inactive.

Comments: Signup button functionality works as expected.

7.3. Testing Phases

- **Development Testing**:
  - Continuous testing during the development phase to catch issues early.
  - Example: Running unit tests and integration tests during each development sprint.
- **Pre-release Testing**:
  - Comprehensive testing before releasing the application to ensure it is ready for production.
  - Example: Conducting thorough UI/UX testing and end-to-end testing.
- **Post-release Testing**:
  - Ongoing testing after the application is released to catch any issues that may arise.
  - Example: Monitoring user feedback and performing regression tests after updates.

8. Results and Discussion

8.1. Expected Results

- **Reduction in Food Waste**:
  - The primary goal is to significantly reduce food waste by facilitating the collection of surplus food from restaurants and its delivery to organic fertilizer factories.
  - Quantitative metrics such as the amount of food waste collected and converted into organic fertilizer can be used to measure success.
- **Environmental Impact**:
  - By converting food waste into organic fertilizer, the project aims to reduce the environmental burden associated with food waste disposal.
  - Metrics like reduced landfill use, decreased greenhouse gas emissions, and increased organic farming outputs can illustrate the environmental benefits.

- **Restaurant Participation**:
  - Increased awareness and participation from restaurants in the circular economy model.
  - The number of participating restaurants and the frequency of their contributions will be key indicators.
- **User Engagement**:
  - The application is designed to be user-friendly and engaging, encouraging users to actively participate in reducing food waste.
  - Metrics such as the number of active users, user retention rates, and user feedback will help gauge engagement levels.

8.2. *Discussion*

- **Impact on United Nations Sustainable Development Goals (SDGs)**:
  - The application aligns with several SDGs, including responsible consumption and production (SDG 12), climate action (SDG 13), and life on land (SDG 15).
  - The impact on these goals can be discussed in terms of how the project contributes to sustainable practices and environmental conservation.
- **Scalability and Replicability**:
  - Discuss the potential for scaling the application to other regions or countries, considering factors such as local regulations, infrastructure, and cultural acceptance.
  - The replicability of the project in different contexts can also be explored.
- **User Feedback and Iterative Improvements**:
  - Highlight the importance of collecting user feedback to continuously improve the application.
  - Discuss the plan for iterative improvements based on user suggestions and technological advancements.
- **Long-term Sustainability**:
  - Consider the long-term sustainability of the project, both in terms of environmental impact and operational viability.
  - Discuss strategies for ensuring the continued success and relevance of the application, such as partnerships, funding opportunities, and community involvement.

9. Conclusion

9.1. Summary

"Food for Good" is a mobile application developed using Flutter, aimed at reducing food waste and promoting environmental sustainability. The application facilitates the collection of wasted food from restaurants and its delivery to organic fertilizer factories, converting potential waste into valuable resources. By leveraging modern mobile

technology, "Food for Good" provides an efficient and scalable solution to the persistent issue of food waste, encouraging restaurants to participate in a circular economy model. This project demonstrates how innovative technological solutions can contribute to sustainable development and environmental conservation.

9.2. Challenges

Throughout the development of the "Food for Good" application, several challenges were encountered:

- **Complex User Flows**: Managing various user roles (restaurant managers, drivers, factory managers) and ensuring a seamless experience for each was challenging.
- **Real-time Data Sync:** Implementing real-time data synchronization for tracking food waste collections required integrating Firebase, which initially posed integration challenges due to its differences from the backend stack utilizing Node.js, Express.js, MongoDB (NoSQL), and RESTful APIs**.**.
- **Validation**: Ensuring robust input validation for forms, especially during user signup and login, was crucial for maintaining data integrity and security.
- **Navigation Management**: Handling navigation between multiple screens while maintaining state and passing data effectively required careful planning and testing.

9.3. Future Work

To further enhance the "Food for Good" application, the following future work is proposed:

- **Expansion of Services**: Extend the application to include more features, such as tracking the entire lifecycle of food waste and providing detailed analytics to users and restaurants.
- **Integration with Additional APIs**: Integrate with more APIs to offer advanced functionalities, such as real-time tracking of food waste deliveries and AI-driven route optimization.
- **Partnerships with More Restaurants and Factories**: Increase collaboration with a larger number of restaurants and organic fertilizer factories to broaden the impact of the application.
- **Selling Organic Products**: Develop a marketplace within the application where users can purchase the organic products generated from the food waste.
- **Enhanced User Experience**: Continuously improve the user interface and user experience based on feedback and technological advancements.
- **Global Scalability**: Adapt the application for global use, taking into account regional differences in food waste management practices and regulations.

- **Educational Campaigns**: Launch educational campaigns through the application to raise awareness about food waste and promote sustainable practices among users and restaurants.

10. References

- Systems Analysis and Design by Alan Dennis, Barbara Haley Wixom, Roberta M. Roth (z-lib.org)
- Project Management JumpStart by Kim Heldman (4th Edition)
- Software Engineering TENTH edition Ian Sommerville
- Software Requirements & Specifications: A Lexicon of Practice, Principles, and Prejudices

11. Appendices

\foodforgood\lib\theme\app_styles.dart

```dart
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class TextStyles {
  static final TextStyle titleStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
  );
  static final TextStyle fieldStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 14, fontWeight: FontWeight.w300),
  );
  static final TextStyle subtitleStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 12, fontWeight: FontWeight.w700),
  );
  static final TextStyle normalStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 12, fontWeight: FontWeight.w300),
  );
  static final TextStyle buttonTextStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.w600),
  );
  static final TextStyle cardTitleStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 20, fontWeight: FontWeight.w500),
```

```
  );
  static final TextStyle cardButtonTextStyle = GoogleFonts.rubik(
    textStyle: const TextStyle(fontSize: 14, fontWeight: FontWeight.w400),
  );
}

class AppColors {
  static const titleColor = Colors.white;
  static const subtitleColor = Colors.white12;
  static const hintTextColor = Colors.white70;
  static const loginColor = Colors.black38;
  static const signupColor = Colors.black38;
  static const buttonBackgroundColor = Colors.white;
  static const buttonTextColor = Colors.teal;
  static const buttonLoginBackgroundColor = Colors.teal;
  static const buttonLoginTextColor = Colors.white;
  static const forgetPasswordColor = Colors.lightBlueAccent;
  static const doYouHaveAccountColor = Colors.white54;
  static const selectedItemColor = Colors.white70;
  static const unselectedItemColor = Colors.black;
  static const confirmButtonTextColor = Colors.white;
  static const confirmButtonBackgroundColor = Colors.teal;
  static const cardBackgroundColor = Colors.black;
  static const cardTitleColor = Colors.white;
  static const buttonAcceptBackgroundColor = Colors.teal;
  static const buttonRejectBackgroundColor = Colors.pink;
  static const cardTextColor = Colors.white70;
}
```

\foodforgood\lib\view\screens\announcement_details_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/custom_button_widget.dart';

import '../../theme/app_styles.dart';

class AnnouncementDetailsScreen extends StatefulWidget {
  const AnnouncementDetailsScreen({super.key});

  @override
  _AnnouncementDetailsScreenState createState() =>
_AnnouncementDetailsScreenState();
}

class _AnnouncementDetailsScreenState extends State<AnnouncementDetailsScreen> {
```

```
final TextEditingController _amountController = TextEditingController();
String _selectedUnit = 'kg';
String _selectedType = 'Organic';

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      centerTitle: true,
      title: Text('Details',
        style: TextStyles.titleStyle,
      ),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const SizedBox(height: 8.0),
          Row(
            children: [
              Expanded(
                flex: 2,
                child: TextField(
                  style: TextStyles.fieldStyle.copyWith(
                    fontWeight: FontWeight.w700
                  ),
                  controller: _amountController,
                  keyboardType: TextInputType.number,
                  decoration: InputDecoration(
                    labelText: 'Amount',
                    labelStyle: TextStyles.fieldStyle.copyWith(
                        fontWeight: FontWeight.w500
                    ),
                    border: const OutlineInputBorder(),
                  ),
                ),
              ),
              const SizedBox(width: 16.0),
              Expanded(
                flex: 1,
                child: DropdownButtonFormField<String>(
                  value: _selectedUnit,
                  decoration: InputDecoration(
                    labelText: 'Unit',
```

```
                labelStyle: TextStyles.fieldStyle.copyWith(
                    fontWeight: FontWeight.w500
                ),
                border: OutlineInputBorder(),
              ),
              onChanged: (value) {
                setState(() {
                  _selectedUnit = value!;
                });
              },
              items: <String>['kg', 'g', 'lb', 'oz']
                  .map<DropdownMenuItem<String>>((String value) {
                return DropdownMenuItem<String>(
                  value: value,
                  child: Text(value,
                    style: TextStyles.fieldStyle.copyWith(
                      fontWeight: FontWeight.w700
                  ),
                  ),
                );
              }).toList(),
            ),
          ),
        ],
      ),
      const SizedBox(height: 16.0),
      DropdownButtonFormField<String>(
        value: _selectedType,
        decoration: InputDecoration(
          labelText: 'Type',
          labelStyle: TextStyles.fieldStyle.copyWith(
              fontWeight: FontWeight.w500
          ),
          border: OutlineInputBorder(),
        ),
        onChanged: (value) {
          setState(() {
            _selectedType = value!;
          });
        },
        items: <String>['Organic', 'Non-Organic']
            .map<DropdownMenuItem<String>>((String value) {
          return DropdownMenuItem<String>(
            value: value,
            child: Text(value,
```

```
                    style: TextStyles.fieldStyle.copyWith(
                    fontWeight: FontWeight.w700
                  ),
                  ),
                );
              }).toList(),
            ),
            const Spacer(),
            SizedBox(
              width: double.infinity,
              child: CustomButtonWidget(
                onButtonPressed: () {
                  // Handle confirm action
                  print('Amount: ${_amountController.text} $_selectedUnit');
                  print('Type: $_selectedType');
                },
                text: 'Confirm',
                textColor: AppColors.buttonLoginTextColor,
                 backgroundColor: AppColors.buttonLoginBackgroundColor,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

\foodforgood\lib\view\screens\driver_orders_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/drivers_orders_widget.dart';

class DriverOrdersScreen extends StatefulWidget {
  const DriverOrdersScreen({super.key});

  @override
  State<DriverOrdersScreen> createState() => _DriverOrdersScreenState();
}

class _DriverOrdersScreenState extends State<DriverOrdersScreen> {
  @override
```

```
  Widget build(BuildContext context) {
    return const DriversOrdersWidget();
  }
}
```

\foodforgood\lib\view\screens\driver_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/driver_start_screen.dart';

class DriverScreen extends StatefulWidget {
  const DriverScreen({super.key});

  @override
  State<DriverScreen> createState() => _DriverScreenState();
}

class _DriverScreenState extends State<DriverScreen> {
  @override
  Widget build(BuildContext context) {
    return const DriverStartScreen();
  }
}
```

foodforgood\lib\view\screens\login_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/theme/app_styles.dart';
import 'package:foodforgood/view/screens/signup_screen.dart';
import 'package:foodforgood/view/widgets/login_form_widget.dart';

import '../widgets/custom_button_widget.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}
```

```
class _LoginScreenState extends State<LoginScreen> {

  void _validateAndSubmit() {
    print("logged in");
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.loginColor,
      body: Center(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const SizedBox(height: 24),
                const _Title("Login"),
                const SizedBox(height: 24),
                LoginFormWidget(onLoginPressed: _validateAndSubmit), // Use the
LoginForm
              ],
            ),
          ),
        ),
      ),
      bottomNavigationBar: const _signUpNavigation(),
    );
  }
}

class _Title extends StatelessWidget {
  final String text;

  const _Title(this.text);

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Text(
        text,
        style: TextStyles.titleStyle.copyWith(color: AppColors.titleColor),
```

```
      ),
    );
  }
}

class _signUpNavigation extends StatelessWidget {
  const _signUpNavigation();

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          GestureDetector(
            onTap: () {
              // Handle sign up navigation
            },
            child: Text(
              "Don't have an account?",
              style: TextStyles.normalStyle.copyWith(color:
AppColors.doYouHaveAccountColor),
            ),
          ),
          const SizedBox(height: 8),
          SizedBox(
            width: double.infinity,
            child: CustomButtonWidget(
              text: 'Sign up',
              onButtonPressed: (){
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (context) => const SignupScreen()),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}
```

/foodforgood\lib\view\screens\resturant_announcment_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/resturant_announcment_widget.dart';

class ResturantAnnouncmentScreen extends StatefulWidget {
  const ResturantAnnouncmentScreen({super.key});

  @override
  State<ResturantAnnouncmentScreen> createState() =>
_ResturantAnnouncmentScreenState();
}

class _ResturantAnnouncmentScreenState extends State<ResturantAnnouncmentScreen>
{
  @override
  Widget build(BuildContext context) {
    return const NewAnnouncementScreen(restaurantName: 'Resturant A',);
  }
}
```

foodforgood\lib\view\screens\signup_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/theme/app_styles.dart';
import 'package:foodforgood/view/screens/driver_screen.dart';
import 'package:foodforgood/view/screens/resturant_announcment_screen.dart';
import 'package:foodforgood/view/widgets/custom_button_widget.dart';
import 'package:foodforgood/view/widgets/scan_docs_screen.dart';

import '../widgets/signup_form_widget.dart';

class SignupScreen extends StatefulWidget {
  const SignupScreen({Key? key}) : super(key: key);

  @override
  _SignupScreenState createState() => _SignupScreenState();
}

class _SignupScreenState extends State<SignupScreen> {
  bool _showSignupForm = true;
```

```
  String? _selectedRole;

  @override
  Widget build(BuildContext context) {
    return WillPopScope(
      onWillPop: () async {
        if (!_showSignupForm) {
          setState(() {
            _showSignupForm = true; // Show SignupFormWidget
          });
          return false; // Prevent default back navigation
        }
        return true; // Allow default back navigation (exit app or previous
screen)
      },
      child: Scaffold(
        backgroundColor: AppColors.signupColor,
        appBar: AppBar(
          backgroundColor: AppColors.signupColor,
          title: _showSignupForm
              ? Center(child: Text('Signup', style:
TextStyles.titleStyle.copyWith(color: AppColors.titleColor)))
              : null, // Hide the title when showing ScanDocScreen
          automaticallyImplyLeading: !_showSignupForm, // Show back button only
when not showing SignupFormWidget
        ),
        body: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(16.0),
            child: _showSignupForm
                ? SignupFormWidget(
              onContinuePressed: (role) {
                setState(() {
                  _selectedRole = role;
                  _showSignupForm = false; // Switch to ScanDocScreen
                });
              },
            )
                : ScanDocScreen(role: _selectedRole),
          ),
        ),
        bottomNavigationBar: !_showSignupForm
            ? Padding(
          padding: const EdgeInsets.all(16.0),
          child: SizedBox(
```

```
            width: double.infinity,
            child: CustomButtonWidget(
              text: 'Sign up',
              onButtonPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (context) => _selectedRole ==
"Driver"? const DriverScreen() : const ResturantAnnouncmentScreen()),
                );
              },
              backgroundColor: AppColors.buttonLoginBackgroundColor,
              textColor: AppColors.buttonLoginTextColor,
            ),
          ),
        )
            : null, // Hide bottom navigation bar when showing SignupFormWidget
      ),
    );
  }
}
```

\foodforgood\lib\view\widgets\custom_button_widget.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/theme/app_styles.dart';

class CustomButtonWidget extends StatelessWidget {
  final String text;
  final Color? backgroundColor;
  final Color? textColor;
  final VoidCallback onButtonPressed;

  const CustomButtonWidget({
    super.key,
    required this.text,
    required this.onButtonPressed,
    this.backgroundColor,
    this.textColor,
  });

  @override
  Widget build(BuildContext context) {
```

```
    return ElevatedButton(
      style: ButtonStyle(
        backgroundColor: MaterialStateProperty.all(backgroundColor ??
AppColors.buttonBackgroundColor),
      ),
      onPressed: onButtonPressed,
      child: Text(
        text,
        style: TextStyles.buttonTextStyle.copyWith(color: textColor ??
AppColors.buttonTextColor),
      ),
    );
  }
}
```

\foodforgood\lib\view\widgets\custom_text_field_widget.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/theme/app_styles.dart';

class CustomTextFieldWidget extends StatefulWidget {
  final TextEditingController controller;
  final String hintText;
  final String? errorText;
  final Function(String)? onChanged;
  final Color? hintColor;
  final Color? textColor;

  const CustomTextFieldWidget({super.key,
    required this.controller,
    required this.hintText,
    this.errorText,
    this.onChanged, this.hintColor, this.textColor,
  });

  @override
  CustomTextFieldWidgetState createState() => CustomTextFieldWidgetState();
}

class CustomTextFieldWidgetState extends State<CustomTextFieldWidget> {
  final FocusNode _focusNode = FocusNode();

  @override
  void initState() {
```

```
    super.initState();
    _focusNode.addListener(() {
      setState(() {});
    });
  }

  @override
  void dispose() {
    _focusNode.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: widget.controller,
      focusNode: _focusNode,
      onChanged: widget.onChanged,
      decoration: InputDecoration(
        errorText: widget.errorText,
        // border: OutlineInputBorder(
        //   borderRadius: BorderRadius.circular(12),
        //   borderSide: BorderSide(
        //     color: widget.errorText != null ? Colors.red : Colors.grey,
        //   ),
        // ),
        hintText: widget.hintText,
        hintStyle: TextStyles.fieldStyle.copyWith(color: widget.hintColor??
Colors.grey),
        // focusedBorder: OutlineInputBorder(
        //   borderRadius: BorderRadius.circular(12),
        //   borderSide: BorderSide(
        //     color: widget.errorText != null ? Colors.red : Colors.blue,
        //   ),
        // ),
        // enabledBorder: OutlineInputBorder(
        //   borderRadius: BorderRadius.circular(12),
        //   borderSide: BorderSide(
        //     color: widget.errorText != null ? Colors.red : Colors.grey,
        //   ),
        // ),
      ),
      style: TextStyles.fieldStyle.copyWith(color: widget.textColor??
Colors.white),
    );
```

```
  }
}
```

\foodforgood\lib\view\widgets\driver_start_screen.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/screens/driver_orders_screen.dart';

import '../../theme/app_styles.dart';

class DriverStartScreen extends StatelessWidget {
  const DriverStartScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        automaticallyImplyLeading: false,
        title: Text('Hi, Sam Harris', style: TextStyles.titleStyle,),
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 32.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text(
                'Are you available to deliver from restaurants to factories right
now?',
                textAlign: TextAlign.center,
                style: TextStyles.subtitleStyle,
              ),
            ],
          ),
        ),
      ),
      bottomNavigationBar: Padding(
        padding: const EdgeInsets.all(16.0),
        child: SizedBox(
          width: 80,
          height: 80,
          child: FloatingActionButton(
            onPressed: () {
```

```
              print('Start delivery');
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const
DriverOrdersScreen()),
              );
            },
            backgroundColor: AppColors.buttonLoginBackgroundColor,
            shape: const CircleBorder(),
            child: Text("Start", style:
TextStyles.buttonTextStyle.copyWith(color: AppColors.buttonLoginTextColor),),
          ),
        ),
      ),
    );
  }
}
```

\foodforgood\lib\view\widgets\drivers_orders_widget.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/order_card_widget.dart';

import '../../theme/app_styles.dart';

class DriversOrdersWidget extends StatelessWidget {
  const DriversOrdersWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        centerTitle: true,
        title: Text(
          'Announcements',
          style: TextStyles.titleStyle,
        ),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
```

```
            const SizedBox(height: 8.0),
            Expanded(
              child: ListView(
                children: [
                  _buildOrderCard(
                    context,
                    'Restaurant A',
                    '123 Main St',
                    5,
                  ),
                  _buildOrderCard(
                    context,
                    'Restaurant B',
                    '456 Elm St',
                    3,
                  ),
                  _buildOrderCard(
                    context,
                    'Restaurant C',
                    '789 Maple Ave',
                    7,
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }

  Widget _buildOrderCard(
      BuildContext context, String restaurantName, String location, int quantity)
{
    return OrderCardWidget(
        restaurantName: restaurantName,
        location: location,
        quantity: quantity,
        onAccept: (){},
        onReject: (){}
    );
  }
}
```

\foodforgood\lib\view\widgets\login_form_widget.dart

```dart
import 'package:flutter/material.dart';
import 'package:foodforgood/theme/app_styles.dart';
import 'package:foodforgood/view/widgets/custom_text_field_widget.dart';
import 'package:foodforgood/view/widgets/password_field_widget.dart';

import 'custom_button_widget.dart';

class LoginFormWidget extends StatefulWidget {
  final VoidCallback onLoginPressed;

  const LoginFormWidget({required this.onLoginPressed, super.key});

  @override
  _LoginFormWidgetState createState() => _LoginFormWidgetState();
}

class _LoginFormWidgetState extends State<LoginFormWidget> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  String? _emailError;
  String? _passwordError;

  @override
  void initState() {
    super.initState();
    _emailController.addListener(_validateEmail);
    _passwordController.addListener(_validatePassword);
  }

  void _validateEmail() {
    setState(() {
      _emailError = _validateEmailText(_emailController.text);
    });
  }

  void _validatePassword() {
    setState(() {
      _passwordError = _validatePasswordText(_passwordController.text);
    });
  }

  String? _validateEmailText(String email) {
```

```
    if (email.isEmpty) {
      return "Email cannot be empty";
    }
    final emailRegex = RegExp(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-
Z]{2,}$");
    if (!emailRegex.hasMatch(email)) {
      return "Enter a valid email";
    }
    return null;
  }

  String? _validatePasswordText(String password) {
    if (password.isEmpty) {
      return "Password cannot be empty";
    }
    if (password.length < 8) {
      return "Password must be at least 8 characters";
    }
    return null;
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const _CustomLabelText("E-mail"),
        const SizedBox(height: 8),
        CustomTextFieldWidget(
          controller: _emailController,
          hintText: "Enter your email",
          errorText: _emailError,
          onChanged: (text) {
            _validateEmail();
          },
        ),
        const SizedBox(height: 16),
        const _CustomLabelText("Password"),
        const SizedBox(height: 8),
        PasswordFieldWidget(
          controller: _passwordController,
          errorText: _passwordError,
          onChanged: (text) {
            _validatePassword();
```

```
          },
        ),
        const SizedBox(height: 16),
        Align(
          alignment: Alignment.centerRight,
          child: GestureDetector(
            onTap: () {
              // Handle forget password logic
            },
            child: Text(
              "Forget password?",
              style: TextStyles.normalStyle.copyWith(color:
AppColors.forgetPasswordColor),
            ),
          ),
        ),
        const SizedBox(height: 24),
        SizedBox(
          width: double.infinity,
          child: CustomButtonWidget(
            text: 'Login',
            onButtonPressed: () {
              _validateEmail();
              _validatePassword();
              if (_emailError == null && _passwordError == null) {
                widget.onLoginPressed();
              }
            },
            backgroundColor: AppColors.buttonLoginBackgroundColor,
            textColor: AppColors.buttonLoginTextColor,
          ),
        ),
      ],
    );
  }
}

class _CustomLabelText extends StatelessWidget {
  final String text;

  const _CustomLabelText(this.text);

  @override
  Widget build(BuildContext context) {
    return Text(
```

```
      text,
      style: TextStyles.subtitleStyle.copyWith(color: AppColors.subtitleColor)
    );
  }
}
```

\foodforgood\lib\view\widgets\order_card_widget.dart

```
import 'package:flutter/material.dart';

import '../../theme/app_styles.dart';

class OrderCardWidget extends StatelessWidget {
  final String restaurantName;
  final String location;
  final int quantity;
  final VoidCallback onAccept;
  final VoidCallback onReject;

  const OrderCardWidget({
    super.key,
    required this.restaurantName,
    required this.location,
    required this.quantity,
    required this.onAccept,
    required this.onReject,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      color: AppColors.cardBackgroundColor,
      margin: const EdgeInsets.symmetric(vertical: 8.0),
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              restaurantName,
              style: TextStyles.cardTitleStyle.copyWith(
                  color: AppColors.cardTitleColor
              )
            ),
```

```
            const SizedBox(height: 8.0),
            Row(
              children: [
                const Icon(Icons.location_on, size: 18.0, color:
AppColors.cardTextColor,),
                const SizedBox(width: 8.0),
                Text(location,
                  style: TextStyles.normalStyle.copyWith(color:
AppColors.cardTextColor)
                  ,),
              ],
            ),
            const SizedBox(height: 8.0),
            Row(
              children: [
                const Icon(Icons.shopping_cart, size: 18.0, color:
AppColors.cardTextColor,),
                const SizedBox(width: 8.0),
                Text('Quantity: $quantity',
                    style: TextStyles.normalStyle.copyWith(color:
AppColors.cardTextColor)
                ),
              ],
            ),
            const SizedBox(height: 16.0),
            Row(
              mainAxisAlignment: MainAxisAlignment.end,
              children: [
                ElevatedButton(
                  onPressed: onAccept,
                  style: ElevatedButton.styleFrom(
                    backgroundColor: AppColors.buttonAcceptBackgroundColor,
                  ),
                  child: Text('Accept',
                    style: TextStyles.cardButtonTextStyle.copyWith(
                      color: AppColors.cardTitleColor
                    ),),
                ),
                const SizedBox(width: 8.0),
                ElevatedButton(
                  onPressed: onReject,
                  style: ElevatedButton.styleFrom(
                    backgroundColor: AppColors.buttonRejectBackgroundColor,
                  ),
                  child: Text('Reject',
```

```
                    style: TextStyles.cardButtonTextStyle.copyWith(
                        color: AppColors.cardTitleColor
                    ),),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

\foodforgood\lib\view\widgets\password_field_widget.dart

```
import 'package:flutter/material.dart';

import '../../theme/app_styles.dart';

class PasswordFieldWidget extends StatefulWidget {
  final TextEditingController controller;
  final String? errorText;
  final Function(String)? onChanged;

  const PasswordFieldWidget({super.key,
    required this.controller,
    this.errorText,
    this.onChanged,
  });

  @override
  PasswordFieldWidgetState createState() => PasswordFieldWidgetState();
}

class PasswordFieldWidgetState extends State<PasswordFieldWidget> {
  bool _obscureText = true;
  final FocusNode _focusNode = FocusNode();

  @override
  void initState() {
    super.initState();
    _focusNode.addListener(() {
```

```
      setState(() {});
    });
  }

  @override
  void dispose() {
    _focusNode.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: widget.controller,
      focusNode: _focusNode,
      obscureText: _obscureText,
      onChanged: widget.onChanged,
      decoration: InputDecoration(
        errorText: widget.errorText,
        hintText: "Enter your password",
        hintStyle: TextStyles.fieldStyle.copyWith(color: Colors.grey),
        suffixIcon: IconButton(
          icon: Icon(
            _obscureText ? Icons.visibility : Icons.visibility_off,
            color: _focusNode.hasFocus ? Colors.blue : Colors.grey,
          ),
          onPressed: () {
            setState(() {
              _obscureText = !_obscureText;
            });
          },
        ),
      ),
      style: TextStyles.fieldStyle.copyWith(color: Colors.white),
    );
  }
}
```

\foodforgood\lib\view\widgets\resturant_announcment_widget.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/screens/announcement_details_screen.dart';
```

```
import '../../theme/app_styles.dart';

class NewAnnouncementScreen extends StatelessWidget {
  final String restaurantName;

  const NewAnnouncementScreen({Key? key, required this.restaurantName}) :
super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        centerTitle: true,
        title: Text(restaurantName,
          style: TextStyles.titleStyle,
        ),
        automaticallyImplyLeading: false,
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // Navigate to the details screen
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const
AnnouncementDetailsScreen()),
            );
          },
          style: ElevatedButton.styleFrom(
            shape: const CircleBorder(),
            padding: const EdgeInsets.all(36.0),
            backgroundColor: AppColors.buttonLoginBackgroundColor,
            minimumSize: const Size(200, 200), // Increase the size of the button
          ),
          child: Text(
            'Set a \nNew Announcement',
            textAlign: TextAlign.center,
            style: TextStyles.buttonTextStyle.copyWith(
              color: AppColors.buttonLoginTextColor
            ),
          ),
        ),
      ),
    );
  }
```

```
}
```

\foodforgood\lib\view\widgets\scan_docs_screen.dart

```dart
import 'package:flutter/material.dart';

import '../../theme/app_styles.dart';

class ScanDocScreen extends StatelessWidget {
  final String? role;

  const ScanDocScreen({Key? key, required this.role}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    String scanButtonText = '';
    String setLocationButtonText = '';

    switch (role) {
      case 'Restaurant Manager':
        scanButtonText = 'Scan Restaurant License';
        setLocationButtonText = 'Set Restaurant Location';
        break;
      case 'Driver':
        scanButtonText = 'Scan Driver License';
        setLocationButtonText = 'Set Driver Location';
        break;
      case 'Factory Manager':
        scanButtonText = 'Scan Factory License';
        setLocationButtonText = 'Set Factory Location';
        break;
    }

    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          SizedBox(
            width: double.infinity,
            child: ElevatedButton(
              onPressed: () {
```

```
                // Handle scanning license based on role
                print(scanButtonText);
              },
              style: ElevatedButton.styleFrom(
                padding: const EdgeInsets.symmetric(horizontal: 16.0, vertical:
12.0),
              ),
              child: Row(
                children: [
                  Expanded(
                    child: Text(
                      scanButtonText,
                      textAlign: TextAlign.left,
                      style: TextStyles.buttonTextStyle.copyWith(color:
Colors.black),
                    ),
                  ),
                  const SizedBox(width: 8.0),
                  const Icon(Icons.camera_alt, color: Colors.black),
                ],
              ),
            ),
          ),
          const SizedBox(height: 16.0),
          SizedBox(
            width: double.infinity,
            child: ElevatedButton(
              onPressed: () {
                // Handle setting location based on role
                print(setLocationButtonText);
              },
              style: ElevatedButton.styleFrom(
                padding: const EdgeInsets.symmetric(horizontal: 16.0, vertical:
12.0),
              ),
              child: Row(
                children: [
                  Expanded(
                    child: Text(
                      setLocationButtonText,
                      textAlign: TextAlign.left,
                      style: TextStyles.buttonTextStyle.copyWith(color:
Colors.black),
                    ),
                  ),
```

```
                  const SizedBox(width: 8.0),
                  const Icon(Icons.location_on, color: Colors.black,),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

/foodforgood\lib\view\widgets\signup_form_widget.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/widgets/custom_text_field_widget.dart';
import 'package:foodforgood/view/widgets/password_field_widget.dart';
import 'package:foodforgood/view/widgets/custom_button_widget.dart';

import '../../theme/app_styles.dart';

enum Gender { Male, Female, Other }

class SignupFormWidget extends StatefulWidget {
  final Function(String?) onContinuePressed;

  const SignupFormWidget({Key? key, required this.onContinuePressed}) :
super(key: key);

  @override
  _SignupFormWidgetState createState() => _SignupFormWidgetState();
}

class _SignupFormWidgetState extends State<SignupFormWidget> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmPasswordController = TextEditingController();
  final _birthdateController = TextEditingController();
  final _firstNameController = TextEditingController();
  final _lastNameController = TextEditingController();
  final _phoneController = TextEditingController();
  final _pinController = TextEditingController();
```

```
Gender _selectedGender = Gender.Male;
String? _selectedRole;

String? _emailError;
String? _firstNameError;
String? _lastNameError;
String? _passwordError;
String? _confirmPasswordError;
String? _birthdateError;
String? _phoneError;
String? _pinError;

bool _isPhoneConfirmed = false;
bool _isSignupButtonActive = false;

@override
void initState() {
  super.initState();
  _firstNameController.addListener(_validateForm);
  _lastNameController.addListener(_validateForm);
  _emailController.addListener(_validateForm);
  _passwordController.addListener(_validateForm);
  _confirmPasswordController.addListener(_validateForm);
  _birthdateController.addListener(_validateForm);
  _phoneController.addListener(_validateForm);
  _firstNameController.addListener(_validateForm);
  _lastNameController.addListener(_validateForm);
}

void _validateFirstName() {
  setState(() {
    _firstNameError = _validateFirstNameText(_firstNameController.text);
  });
}

void _validateLastName() {
  setState(() {
    _lastNameError = _validateLastNameText(_lastNameController.text);
  });
}

void _validateEmail() {
  setState(() {
    _emailError = _validateEmailText(_emailController.text);
  });
```

```
}

void _validatePassword() {
  setState(() {
    _passwordError = _validatePasswordText(_passwordController.text);
  });
}

void _validateConfirmPassword() {
  setState(() {
    _confirmPasswordError = _validateConfirmPasswordText(
      _passwordController.text,
      _confirmPasswordController.text,
    );
  });
}

void _validateBirthdate() {
  setState(() {
    _birthdateError = _validateBirthdateText(_birthdateController.text);
  });
}

void _validatePhone() {
  setState(() {
    _phoneError = _validatePhoneText(_phoneController.text);
  });
}

void _validateForm() {
  setState(() {
    _emailError = _validateEmailText(_emailController.text);
    _passwordError = _validatePasswordText(_passwordController.text);
    _confirmPasswordError = _validateConfirmPasswordText(
      _passwordController.text,
      _confirmPasswordController.text,
    );
    _birthdateError = _validateBirthdateText(_birthdateController.text);
    _phoneError = _validatePhoneText(_phoneController.text);
    _firstNameError = _validateFirstNameText(_firstNameController.text);
    _lastNameError = _validateLastNameText(_lastNameController.text);

    _isSignupButtonActive = _emailError == null &&
        _passwordError == null &&
        _firstNameError == null &&
```

```
          _lastNameError == null &&
          _confirmPasswordError == null &&
          _birthdateError == null &&
          _phoneError == null &&
          _firstNameController.text.isNotEmpty &&
          _lastNameController.text.isNotEmpty &&
          _selectedRole != null &&
          _pinController.text == '0000' &&
          _isPhoneConfirmed == true;
    });
  }

  String? _validateEmailText(String email) {
    if (email.isEmpty) {
      return "Email cannot be empty";
    }
    final emailRegex = RegExp(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-
Z]{2,}$");
    if (!emailRegex.hasMatch(email)) {
      return "Enter a valid email";
    }
    return null;
  }

  String? _validateFirstNameText(String first) {
    if (first.isEmpty) {
      return "Name cannot be empty";
    }
    return null;
  }

  String? _validateLastNameText(String last) {
    if (last.isEmpty) {
      return "Name cannot be empty";
    }
    return null;
  }

  String? _validatePasswordText(String password) {
    if (password.isEmpty) {
      return "Password cannot be empty";
    }
    if (password.length < 8) {
      return "Password must be at least 8 characters";
    }
```

```
    return null;
  }

  String? _validateConfirmPasswordText(String password, String confirmPassword) {
    if (confirmPassword.isEmpty) {
      return "Confirm password cannot be empty";
    }
    if (password != confirmPassword) {
      return "Passwords do not match";
    }
    return null;
  }

  String? _validateBirthdateText(String birthdate) {
    if (birthdate.isEmpty) {
      return "Birthdate cannot be empty";
    }
    final birthdateRegex = RegExp(r"^(0[1-9]|1[0-2])/(0[1-9]|[12][0-9]|3[01])/[0-
9]{4}$");
    if (!birthdateRegex.hasMatch(birthdate)) {
      return "Enter a valid birthdate in MM/DD/YYYY format";
    }
    return null;
  }

  String? _validatePhoneText(String phone) {
    if (phone.isEmpty) {
      return "Phone number cannot be empty";
    }
    final phoneRegex = RegExp(r"^\d{11}$");
    if (!phoneRegex.hasMatch(phone)) {
      return "Enter a valid phone number";
    }
    return null;
  }

  void _handleSignup() {
    if (_isSignupButtonActive) {
      widget.onContinuePressed(_selectedRole);
    }
  }

  void _showPinInputModal() {
    showModalBottomSheet(
      context: context,
```

```
      isScrollControlled: true,
      builder: (BuildContext context) {
        return Padding(
          padding: EdgeInsets.only(
            bottom: MediaQuery.of(context).viewInsets.bottom,
          ),
          child: SingleChildScrollView(
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text(
                    'Enter the PIN sent to your phone',
                    style: TextStyles.subtitleStyle.copyWith(color:
AppColors.signupColor),
                  ),
                  const SizedBox(height: 16),
                  CustomTextFieldWidget(
                    controller: _pinController,
                    hintText: 'Enter PIN',
                    errorText: _pinError,
                    onChanged: (text) {
                      _validatePin();
                    },
                    textColor: Colors.black,
                  ),
                  const SizedBox(height: 16),
                  SizedBox(
                    width: double.infinity,
                    child: CustomButtonWidget(
                      text: 'Submit',
                      onButtonPressed: _validatePin,
                      backgroundColor: AppColors.buttonLoginBackgroundColor,
                      textColor: AppColors.buttonLoginTextColor,
                    ),
                  ),
                ],
              ),
            ),
          ),
        );
      },
    );
  }
```

```
void _validatePin() {
  setState(() {
    _pinError = _validatePinText(_pinController.text);
  });

  if (_pinError == null) {
    setState(() {
      _isPhoneConfirmed = true;
    });
    Navigator.pop(context); // Close the bottom sheet
  }
}

String? _validatePinText(String pin) {
  if (pin.isEmpty) {
    return "PIN cannot be empty";
  }
  if (pin != '0000') {
    return "Invalid PIN";
  }
  return null;
}

@override
Widget build(BuildContext context) {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const _CustomLabelText("First Name"),
                const SizedBox(height: 8),
                CustomTextFieldWidget(
                  controller: _firstNameController,
                  hintText: "First name",
                  errorText: _firstNameError,
                  onChanged: (text) {
                    _validateFirstName();
                  },
```

```
                ),
              ],
            ),
          ),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const _CustomLabelText("Last Name"),
                const SizedBox(height: 8),
                CustomTextFieldWidget(
                  controller: _lastNameController,
                  hintText: "Last name",
                  errorText: _lastNameError,
                  onChanged: (text) {
                    _validateLastName();
                  },
                ),
              ],
            ),
          ),
        ],
      ),
      const SizedBox(height: 16),
      const _CustomLabelText("E-mail"),
      const SizedBox(height: 8),
      CustomTextFieldWidget(
        controller: _emailController,
        hintText: "Enter your email",
        errorText: _emailError,
        onChanged: (text) {
          _validateEmail();
        },
      ),
      const SizedBox(height: 16),
      const _CustomLabelText("Password"),
      const SizedBox(height: 8),
      PasswordFieldWidget(
        controller: _passwordController,
        errorText: _passwordError,
        onChanged: (text) {
          _validatePassword();
        },
      ),
```

```
        const SizedBox(height: 16),
        const _CustomLabelText("Confirm Password"),
        const SizedBox(height: 8),
        PasswordFieldWidget(
          controller: _confirmPasswordController,
          errorText: _confirmPasswordError,
          onChanged: (text) {
            _validateConfirmPassword();
          },
        ),
        const SizedBox(height: 16),
        Row(
          children: [
            Expanded(
              flex: 4,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const _CustomLabelText("Birthdate"),
                  const SizedBox(height: 8),
                  CustomTextFieldWidget(
                    controller: _birthdateController,
                    errorText: _birthdateError,
                    hintText: "MM/DD/YYYY",
                    onChanged: (text) {
                      _validateBirthdate();
                    },
                  ),
                ],
              ),
            ),
            const SizedBox(width: 16),
            Expanded(
              flex: 2,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const _CustomLabelText("Gender"),
                  const SizedBox(height: 8),
                  DropdownButton<Gender>(
                    isExpanded: true,
                    value: _selectedGender,
                    onChanged: (value) {
                      setState(() {
                        _selectedGender = value!;
```

```
                    });
                  },
                  items: [
                    DropdownMenuItem(
                      value: Gender.Male,
                      child: Text("Male",
                        style: _selectedGender == Gender.Male?
                        TextStyles.fieldStyle.copyWith(color:
AppColors.selectedItemColor) :
                        TextStyles.fieldStyle.copyWith(color:
AppColors.unselectedItemColor)),
                    ),
                    DropdownMenuItem(
                      value: Gender.Female,
                      child: Text("Female", style: _selectedGender ==
Gender.Female?
                        TextStyles.fieldStyle.copyWith(color:
AppColors.selectedItemColor) :
                        TextStyles.fieldStyle.copyWith(color:
AppColors.unselectedItemColor)),
                    ),
                    DropdownMenuItem(
                      value: Gender.Other,
                      child: Text("Other", style: _selectedGender ==
Gender.Other?
                        TextStyles.fieldStyle.copyWith(color:
AppColors.selectedItemColor) :
                        TextStyles.fieldStyle.copyWith(color:
AppColors.unselectedItemColor)),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 16),
        Row(
          crossAxisAlignment: CrossAxisAlignment.end,
          children: [
            Expanded(
              flex: 4,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
```

```
                children: [
                  const _CustomLabelText("Phone Number"),
                  const SizedBox(height: 8),
                  CustomTextFieldWidget(
                    controller: _phoneController,
                    hintText: "Enter your phone number",
                    errorText: _phoneError,
                    onChanged: (text) {
                      _validatePhone();
                    },
                  ),
                ],
              ),
            ),
            const SizedBox(width: 16),
            Expanded(
              flex: 2,
              child: Align(
                alignment: Alignment.bottomCenter,
                child: Column(
                  children: [
                    SizedBox(
                      width: double.infinity,
                      height: 48.0,
                      child: ElevatedButton(
                        onPressed: () {
                          _showPinInputModal();
                        },
                        style: ElevatedButton.styleFrom(
                          shape: RoundedRectangleBorder(
                            borderRadius: BorderRadius.circular(4.0),
                          ),
                          backgroundColor:
AppColors.confirmButtonBackgroundColor,
                        ),
                        child: Text("Confirm",
                          style: TextStyles.subtitleStyle.copyWith(color:
AppColors.confirmButtonTextColor),),
                      ),
                    ),
                    if (_phoneError != null)
                      const SizedBox(height: 26.0),
                  ],
                ),
              ),
```

```
            ),
          ],
        ),
        const SizedBox(height: 16),
        SizedBox(
          width: double.infinity,
          child: DropdownButton<String>(
            value: _selectedRole,
            hint: Text('Choose a role', style:
TextStyles.fieldStyle.copyWith(color: AppColors.hintTextColor),),
            onChanged: (String? newValue) {
              setState(() {
                _selectedRole = newValue;
              });
              _validateForm(); // Validate form whenever role is changed
            },
            icon: const Icon(Icons.arrow_drop_down),
            isExpanded: true,
            items: <String>['Restaurant Manager', 'Driver', 'Factory Manager']
                .map<DropdownMenuItem<String>>((String value) {
              return DropdownMenuItem<String>(
                value: value,
                child: Text(value, style: _selectedRole == value?
                TextStyles.fieldStyle.copyWith(color:
AppColors.selectedItemColor) :
                TextStyles.fieldStyle.copyWith(color:
AppColors.unselectedItemColor)),
              );
            }).toList(),
          ),
        ),
        const SizedBox(height: 24),
        SizedBox(
          width: double.infinity,
          child: CustomButtonWidget(
            onButtonPressed: () {
              _validateForm();
              if (_isSignupButtonActive) {
                _handleSignup();
              }
              },
            text: "Continue",
          ),
        ),
      ],
```

```
    );
  }
}

class _CustomLabelText extends StatelessWidget {
  final String text;

  const _CustomLabelText(this.text);

  @override
  Widget build(BuildContext context) {
    return Text(
      text,
      style: TextStyles.subtitleStyle.copyWith(color: AppColors.subtitleColor),
    );
  }
}
```

\foodforgood\lib\main.dart

```
import 'package:flutter/material.dart';
import 'package:foodforgood/view/screens/login_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {

  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: LoginScreen(),
    );
  }
}
```